

Lecture 7: Asynchronous programming

Presentations students, introduction to async

Willem Vanhulle

DevLab Rust 2025

Tuesday December 9, 2025

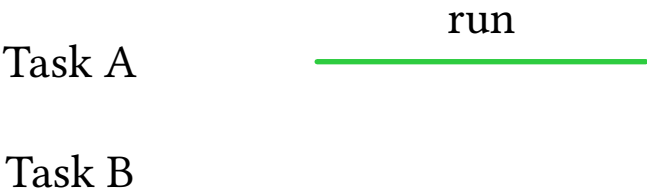
github.com/wvhulle/rust-course-ghent

1. Student presentations	1
1.1. Time-series transformers (Wietse)	2
1.2. Resource pool (Thomas)	3
2. Asynchronous programming	4
3. Asynchronous Rust	8
4. Rust's async implementation	15
5. Ecosystem	22
6. Common async datatypes and functions	28
7. Streams	38
8. Pitfalls	44
9. Exercise: async philosophers	49
10. Exercise: broadcast chat application	53

1. Student presentations	1
2. Asynchronous programming	4
2.1. Introduction	5
2.2. Comparison with Python	6
2.3. Comparison with C++	7
3. Asynchronous Rust	8
4. Rust's async implementation	15
5. Ecosystem	22
6. Common async datatypes and functions	28
7. Streams	38
8. Pitfalls	44
9. Exercise: async philosophers	49
10. Exercise: broadcast chat application	53

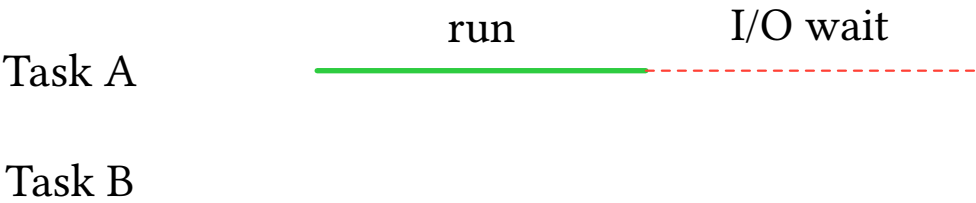
2.1. Introduction

Multiple tasks execute concurrently by running until they would block on I/O, then yielding to another ready task.

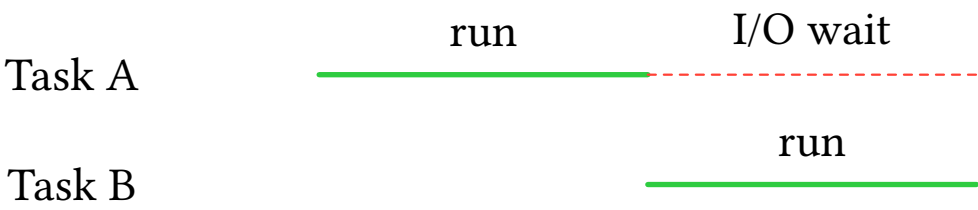


2.1. Introduction

Multiple tasks execute concurrently by running until they would block on I/O, then yielding to another ready task.

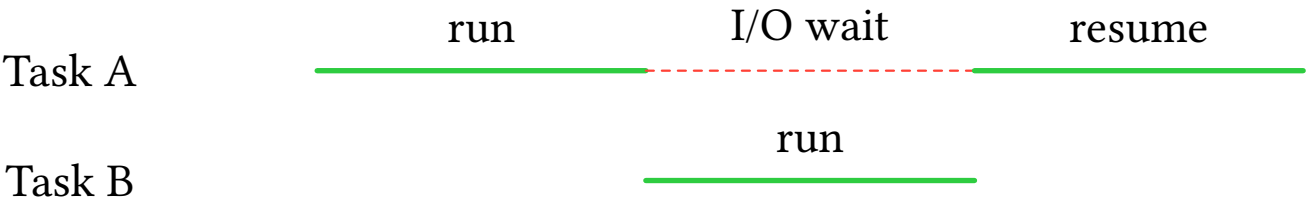


Multiple tasks execute concurrently by running until they would block on I/O, then yielding to another ready task.



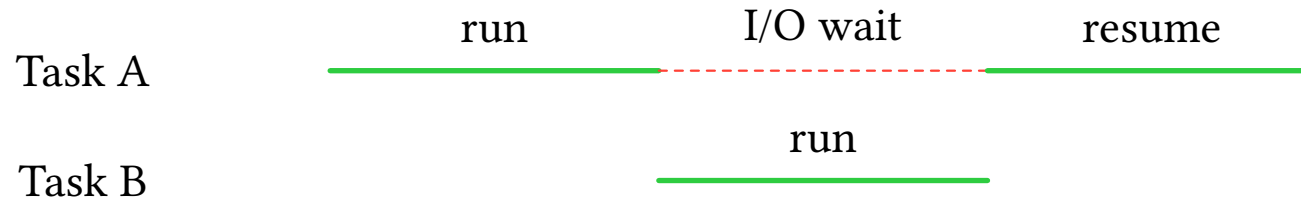
2.1. Introduction

Multiple tasks execute concurrently by running until they would block on I/O, then yielding to another ready task.

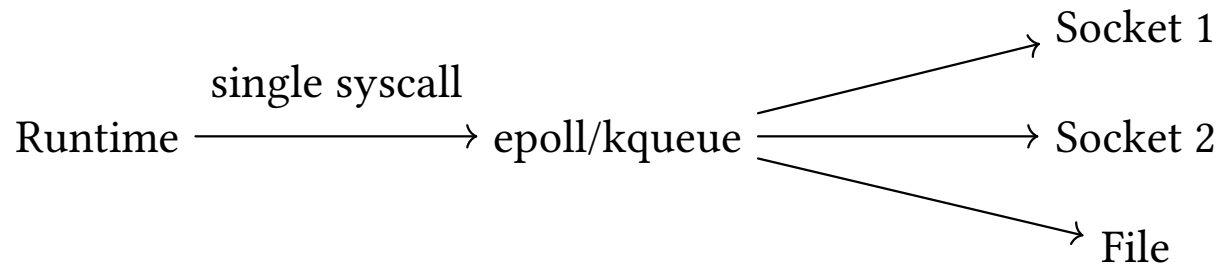


2.1. Introduction

Multiple tasks execute concurrently by running until they would block on I/O, then yielding to another ready task.

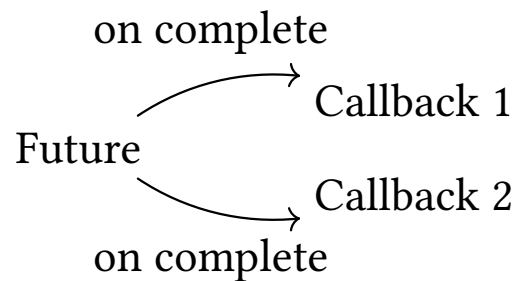


Per-task overhead is minimal because operating systems provide primitives (epoll, kqueue, IOCP) that monitor many I/O operations with a single system call.



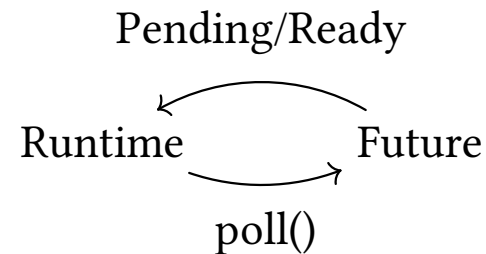
2.2. Comparison with Python

Python asyncio



- Callback-based model
- Register callbacks
- Built-in event loop
- Interpreter overhead

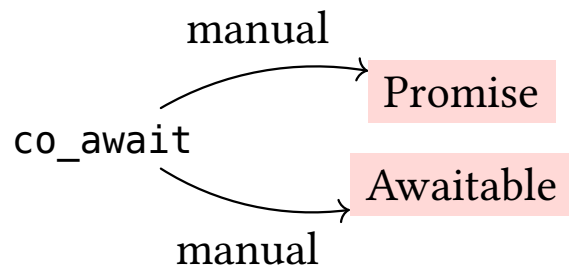
Rust async/await



- Poll-based model
- Runtime polls futures
- External runtimes (tokio, async-std)
- Zero-cost state machines

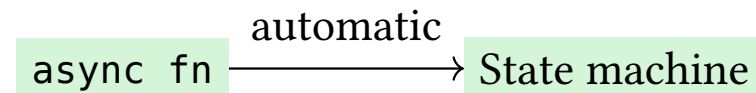
2.3. Comparison with C++

C++20 Coroutines



- Requires custom promise and awaitable types
- Low-level control over suspension
- Flexible but verbose
- Coroutine frame heap-allocated (optimizable)

Rust `async/await`



- Compiler automatically generates Future trait
- High-level syntax with `.await`
- Ecosystem provides runtime and utilities
- State machine stack-allocated by default

1. Student presentations	1
2. Asynchronous programming	4
3. Asynchronous Rust	8
3.1. Await syntax	9
3.2. Futures	10
3.3. Simplest Future	11
3.4. More realistic JoinHandle	12
3.5. Spawning	13
3.6. Avoid spawn-ception	14
4. Rust's async implementation	15
5. Ecosystem	22
6. Common async datatypes and functions	28
7. Streams	38
8. Pitfalls	44
9. Exercise: async philosophers	49
10. Exercise: broadcast chat application	53

3.1. Await syntax

At a high level, async Rust code looks very much like “normal” sequential code:

```
1  use futures::executor::block_on;
2
3  async fn count_to(count: i32) {
4      for i in 0..count {
5          println!("Count is: {i}!");
6      }
7  }
8
9  async fn async_main(count: i32) {
10     count_to(count).await;
11 }
12
13 fn main() {
14     block_on(async_main(10));
15 }
```

([playground link](#))

3.2. Futures

Future is a trait, implemented by objects that represent an operation that may not be complete yet. A future can be polled, and poll returns a Poll.

```
1  use std::pin::Pin; use std::task::Context;
2
3  pub trait Future {
4      type Output;
5      fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
6  }
7
8  pub enum Poll<T> {
9      Ready(T),
10     Pending,
11 }
```

([playground link](#))

What is the role of Context?

3.2. Futures

Future is a trait, implemented by objects that represent an operation that may not be complete yet. A future can be polled, and poll returns a Poll.

```
1  use std::pin::Pin; use std::task::Context;
2
3  pub trait Future {
4      type Output;
5      fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
6  }
7
8  pub enum Poll<T> {
9      Ready(T),
10     Pending,
11 }
```

[\(playground link\)](#)

What is the role of Context?

Context allows a Future to schedule itself to be polled again when an event such as a timeout occurs.

3.3. Simplest Future

The simplest future is an Option.

```
1  #[must_use = "futures do nothing unless you `.await` or poll them"]
2  pub struct Ready<T>(Option<T>);
3
4  impl<T> Future for Ready<T> {
5      type Output = T;
6
7      fn poll(mut self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<T> {
8          Poll::Ready(self.0.take().expect("Ready polled after completion"))
9      }
10 }
```

[\(playground link\)](#)

3.3. Simplest Future

The simplest future is an Option.

```
1  #[must_use = "futures do nothing unless you `.await` or poll them"]
2  pub struct Ready<T>(Option<T>);
3
4  impl<T> Future for Ready<T> {
5      type Output = T;
6
7      fn poll(mut self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<T> {
8          Poll::Ready(self.0.take().expect("Ready polled after completion"))
9      }
10 }
```

[\(playground link\)](#)

Not a very interesting future of course!

Name a few “real” (also called leaf) futures

3.3. Simplest Future

The simplest future is an Option.

```
1  #[must_use = "futures do nothing unless you `.await` or poll them"]
2  pub struct Ready<T>(Option<T>);
3
4  impl<T> Future for Ready<T> {
5      type Output = T;
6
7      fn poll(mut self: Pin<&mut Self>, _cx: &mut Context<'_>) -> Poll<T> {
8          Poll::Ready(self.0.take().expect("Ready polled after completion"))
9      }
10 }
```

[\(playground link\)](#)

Not a very interesting future of course!

Name a few “real” (also called leaf) futures

...

3.4. More realistic JoinHandle

One common type implementing Future is a Tokio JoinHandle:

```
1 use tokio::task;  
2  
3 let join = task::spawn(async {  
4     panic!("something bad happened!")  
5 });  
6  
7 // The returned result indicates that the task failed.  
8 assert!(join.await.is_err());
```

[\(playground link\)](#)

3.5. Spawning

Spawning creates a new task that runs concurrently with the spawner.

Use spawning when tasks can progress independently:

```
1  async fn main() {  
2      let listener = TcpListener::bind("127.0.0.1:8080").await.unwrap();  
3      loop {  
4          let (socket, addr) = listener.accept().await.unwrap();  
5          // Spawn a new task for each connection  
6          tokio::spawn(async move {  
7              handle_client(socket, addr).await;  
8          });  
9      }  
10 }
```

[\(playground link\)](#)

3.5. Spawning

Spawning creates a new task that runs concurrently with the spawner.

Use spawning when tasks can progress independently:

```
1  async fn main() {  
2      let listener = TcpListener::bind("127.0.0.1:8080").await.unwrap();  
3      loop {  
4          let (socket, addr) = listener.accept().await.unwrap();  
5          // Spawn a new task for each connection  
6          tokio::spawn(async move {  
7              handle_client(socket, addr).await;  
8          });  
9      }  
10 }
```

[\(playground link\)](#)

Each connection is handled independently without blocking the server from accepting new connections.



3.6. Avoid spawn-ception

Warning

Spawning tasks from within spawned tasks creates hard-to-debug complexity.

Common anti-pattern:

```
1 tokio::spawn(async {  
2     // Some work  
3     tokio::spawn(async { // Nested spawn!  
4         // More work  
5         tokio::spawn(async { /* ... */ })  
6     })  
7 })
```

[*\(playground link\)*](#)

3.6. Avoid spawn-ception

Warning

Spawning tasks from within spawned tasks creates hard-to-debug complexity.

Common anti-pattern:

```
1 tokio::spawn(async {  
2     // Some work  
3     tokio::spawn(async { // Nested spawn!  
4         // More work  
5         tokio::spawn(async { /* ... */ })  
6     })  
7 })
```

[\(playground link\)](#)

Recommendation: Spawn once at application boundaries:

- Spawn tasks at the top level (e.g., per incoming request)
- Use regular async functions and `.await` for sequential steps
- Only spawn when you need true concurrent execution

1.	Student presentations	1
2.	Asynchronous programming	4
3.	Asynchronous Rust	8
4.	Rust's async implementation	15
4.1.	State machine	16
4.2.	Stackless vs Stackful	17
4.3.	Transformation	18
4.4.	State machine structure	19
4.5.	Recursion	20
4.6.	Coroutines	21
5.	Ecosystem	22
6.	Common async datatypes and functions	28
7.	Streams	38
8.	Pitfalls	44
9.	Exercise: async philosophers	49
10.	Exercise: broadcast chat application	53

◁packt▷



1ST EDITION

Asynchronous Programming in Rust

Learn asynchronous programming by building working
examples of futures, green threads, and runtimes

CARL FREDRIK SAMSON

4.1. State machine

Rust transforms async functions into state machines that implement `Future`.

4.1. State machine

Rust transforms async functions into state machines that implement Future.

Key characteristics:

- Calling an async function constructs and returns a future
- Does not execute immediately (lazy)
- Tracks progress through suspension points

4.1. State machine

Rust transforms async functions into state machines that implement Future.

Key characteristics:

- Calling an async function constructs and returns a future
- Does not execute immediately (lazy)
- Tracks progress through suspension points

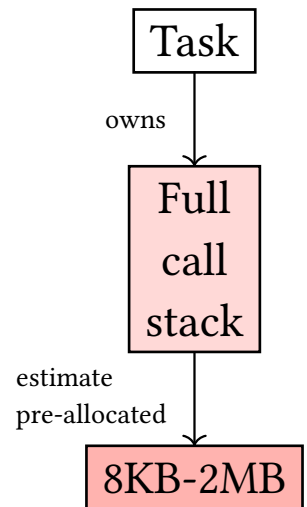
The state machines are allocated on the stack by default. Once allocated, the async runtime never moves the state machine to a different memory location.

Definition

Rust uses **stackless coroutines**: async functions don't have their own call stack. Instead, they compile to state machines that store only the data needed at suspension points, enabling zero-cost abstractions.

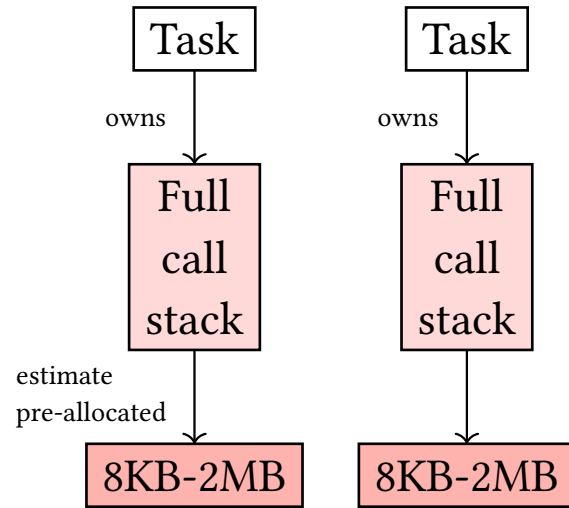
4.2. Stackless vs Stackful

Stackful (Go, goroutines)



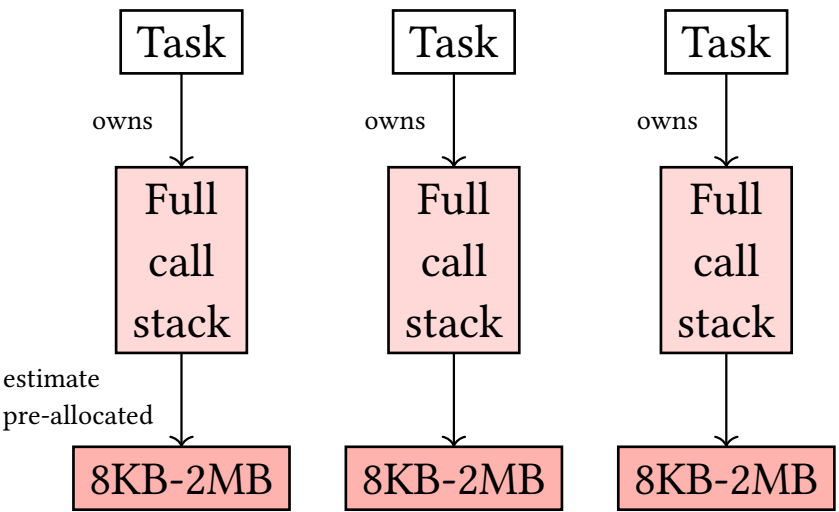
4.2. Stackless vs Stackful

Stackful (Go, goroutines)



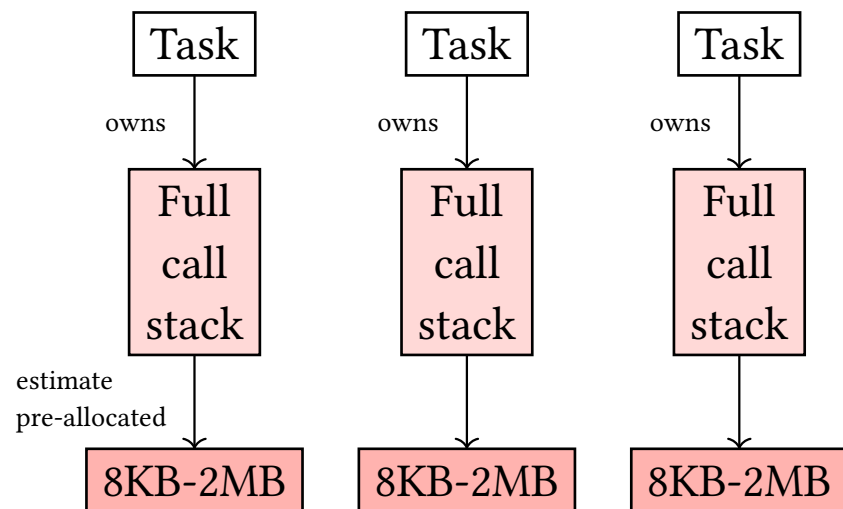
4.2. Stackless vs Stackful

Stackful (Go, goroutines)



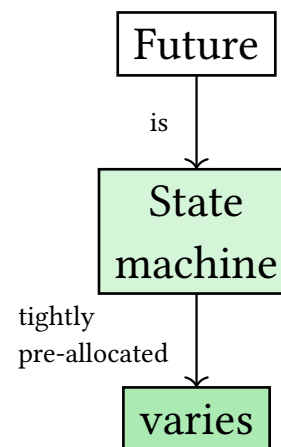
4.2. Stackless vs Stackful

Stackful (Go, goroutines)



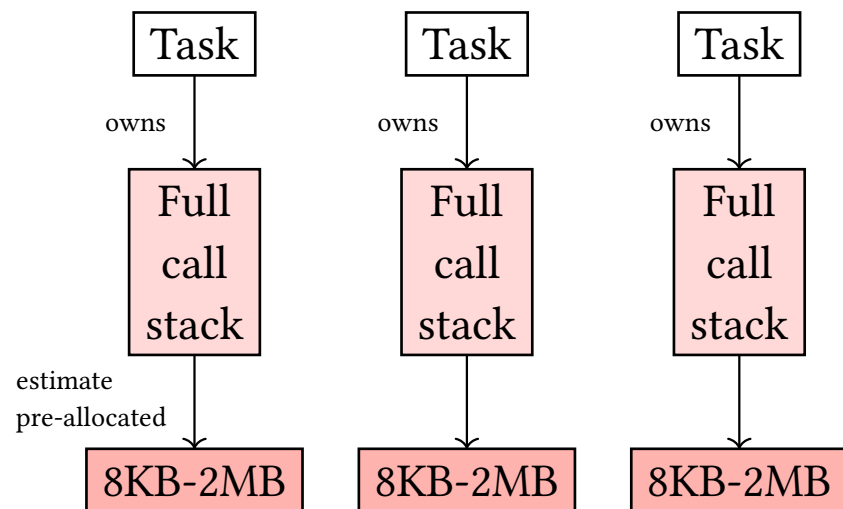
10K tasks = 80MB-20GB

Stackless (Rust async)



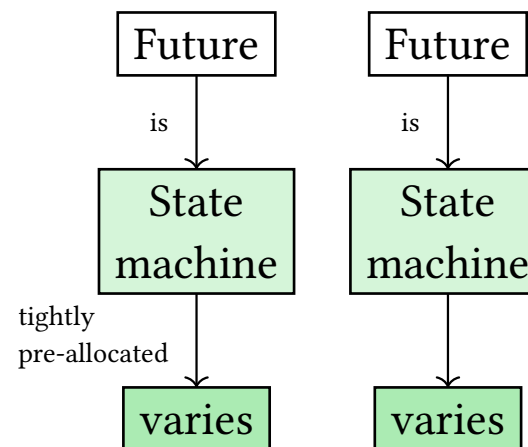
4.2. Stackless vs Stackful

Stackful (Go, goroutines)



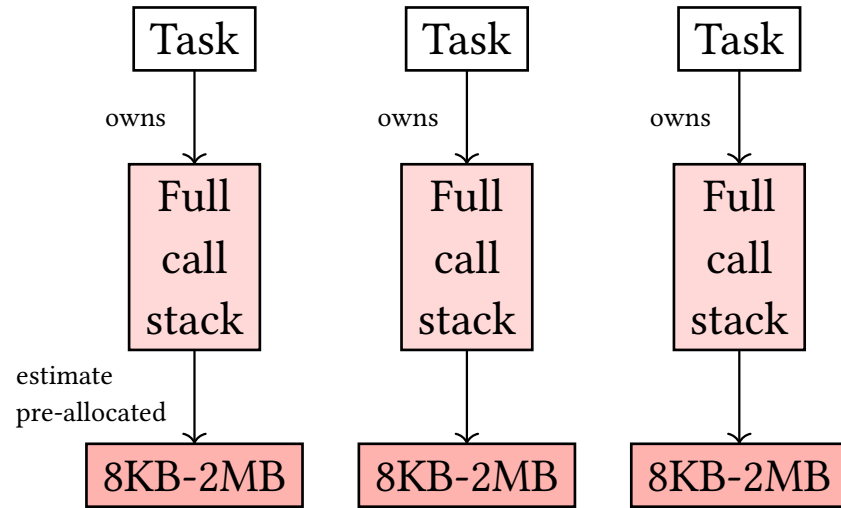
10K tasks = 80MB-20GB

Stackless (Rust async)



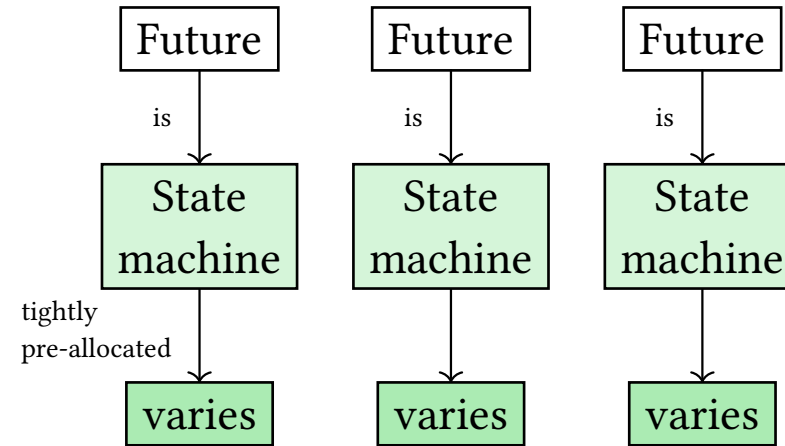
4.2. Stackless vs Stackful

Stackful (Go, goroutines)



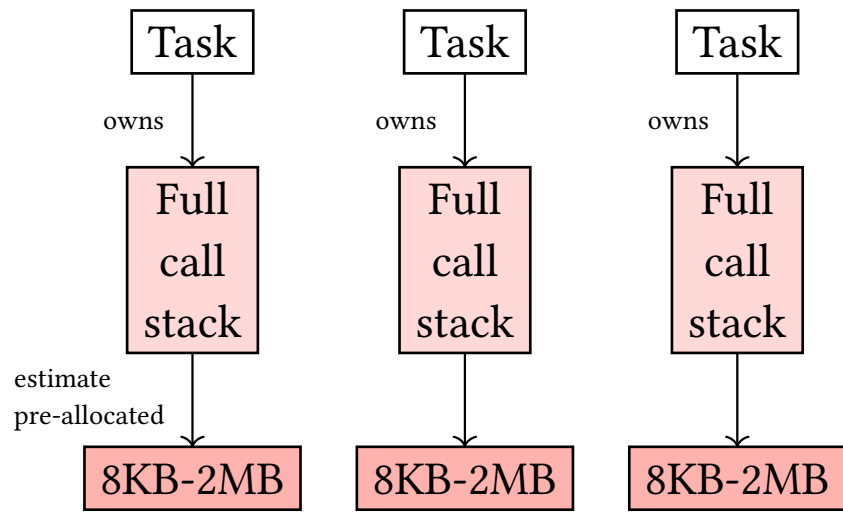
10K tasks = 80MB-20GB

Stackless (Rust async)



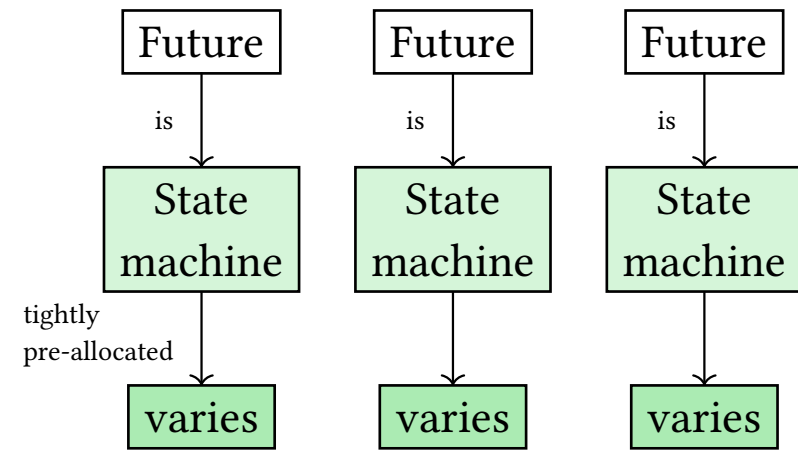
4.2. Stackless vs Stackful

Stackful (Go, goroutines)



10K tasks = 80MB-20GB

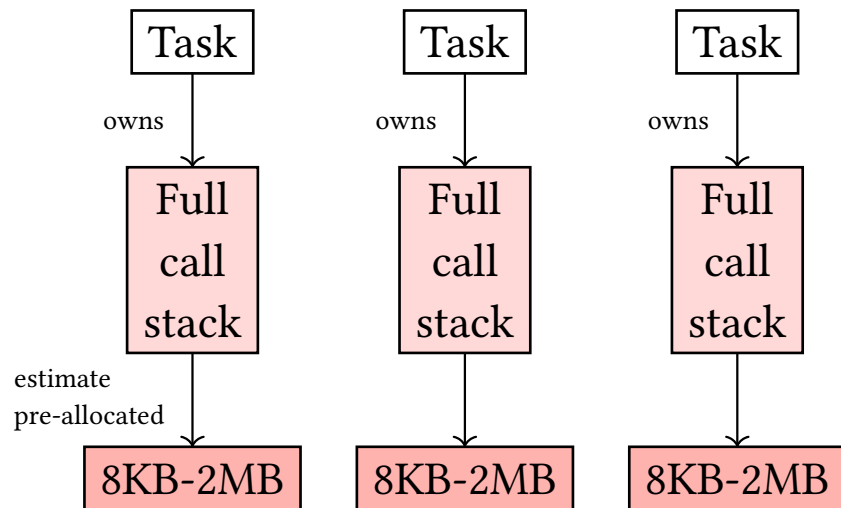
Stackless (Rust async)



10K tasks = 1MB

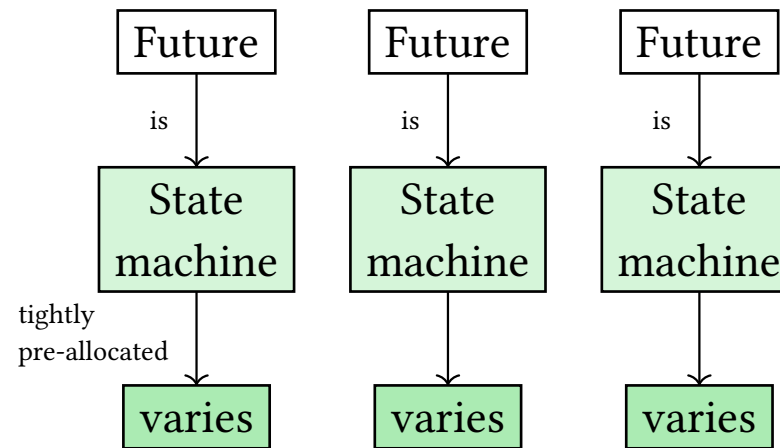
4.2. Stackless vs Stackful

Stackful (Go, goroutines)



10K tasks = 80MB-20GB

Stackless (Rust async)



10K tasks = 1MB

Key difference: Stackful tasks allocate a fixed-size stack (pre-allocated for deep call chains), while stackless state machines allocate only the exact space needed for live variables at each suspension point.

4.3. Transformation

Consider this async function:

```
1 async fn two_d10(modifier: u32) -> u32 {  
2     let first_roll = roll_d10().await;  
3     let second_roll = roll_d10().await;  
4     first_roll + second_roll + modifier  
5 }
```

[\(playground link\)](#)

4.3. Transformation

Consider this async function:

```
1 async fn two_d10(modifier: u32) -> u32 {  
2     let first_roll = roll_d10().await;  
3     let second_roll = roll_d10().await;  
4     first_roll + second_roll + modifier  
5 }
```

[\(playground link\)](#)

The compiler generates an enum that:

- Tracks each suspension point (each `.await`)
- Stores all live variables needed at that state
- Implements the Future trait

4.4. State machine structure

The generated enum looks conceptually like this:

```
1 enum TwoDiceFuture {  
2     Start { modifier: u32 },  
3     FirstRoll { modifier: u32, first_roll: impl Future<Output = u32> },  
4     SecondRoll { modifier: u32, first_result: u32, second_roll: impl Future<Output =  
5         u32> },  
6     Done,  
7 }
```

[\(playground link\)](#)

4.4. State machine structure

The generated enum looks conceptually like this:

```
1 enum TwoDiceFuture {  
2     Start { modifier: u32 },  
3     FirstRoll { modifier: u32, first_roll: impl Future<Output = u32> },  
4     SecondRoll { modifier: u32, first_result: u32, second_roll: impl Future<Output =  
5         u32> },  
6     Done,  
}
```

[\(playground link\)](#)

Each state stores:

- The variables still needed
- The future being awaited (if any)

Deeply nested async functions create large compiler-generated Future types.

Each function's Future contains its callees' Futures.

4.5. Recursion

Deeply nested async functions create large compiler-generated Future types.

Each function's Future contains its callees' Futures.

Recursive async functions require boxing to avoid infinite type sizes:

```
1  async fn count_to(n: u32) {  
2      if n > 0 {  
3          Box::pin(count_to(n - 1)).await;  
4          println!("{n}");  
5      }  
6  }  
7  
8  Box::pin(count_to(n - 1)).await;
```

[\(playground link\)](#)

4.5. Recursion

Deeply nested async functions create large compiler-generated Future types.

Each function's Future contains its callees' Futures.

Recursive async functions require boxing to avoid infinite type sizes:

```
1 async fn count_to(n: u32) {  
2     if n > 0 {  
3         Box::pin(count_to(n - 1)).await;  
4         println!("{n}");  
5     }  
6 }  
7  
8 Box::pin(count_to(n - 1)).await;
```

([playground link](#))

This adds heap allocation overhead but makes recursion possible.

4.6. Coroutines

Async functions in Rust are coroutines that yield nothing.

Definition

Coroutines are a type of functions that may yield intermediate results and can be resumed.

```
1 fn main() {  
2     let mut coroutine = #[coroutine] || {  
3         yield 1;  
4         return "foo"  
5     };  
6  
7     match Pin::new(&mut coroutine).resume(()) {  
8         CoroutineState::Yielded(1) => {}  
9         _ => panic!("unexpected value from resume"),  
10    }  
11    match Pin::new(&mut coroutine).resume(()) {  
12        CoroutineState::Complete("foo") => {}  
13        _ => panic!("unexpected value from resume"),  
14    }  
15 }
```

[\(playground link\)](#)

1.	Student presentations	1
2.	Asynchronous programming	4
3.	Asynchronous Rust	8
4.	Rust's async implementation	15
5.	Ecosystem	22
5.1.	Async runtimes	23
5.2.	Tokio	24
5.3.	Tasks	25
5.4.	Exercise	27
6.	Common async datatypes and functions	28
7.	Streams	38
8.	Pitfalls	44
9.	Exercise: async philosophers	49
10.	Exercise: broadcast chat application	53

5.1. Async runtimes

A runtime provides support for:

- **reactor**: reacting to IO events asynchronously
- **executor**: pushing futures forward concurrently

5.1. Async runtimes

A runtime provides support for:

- **reactor**: reacting to IO events asynchronously
- **executor**: pushing futures forward concurrently

Rust does not have a “built-in” runtime, but several options are available:

- futures offers a bare bones executor
ThreadPool
- tokio offers a Runtime with reactor included
(supplies “time” etc.)

5.1. Async runtimes

A runtime provides support for:

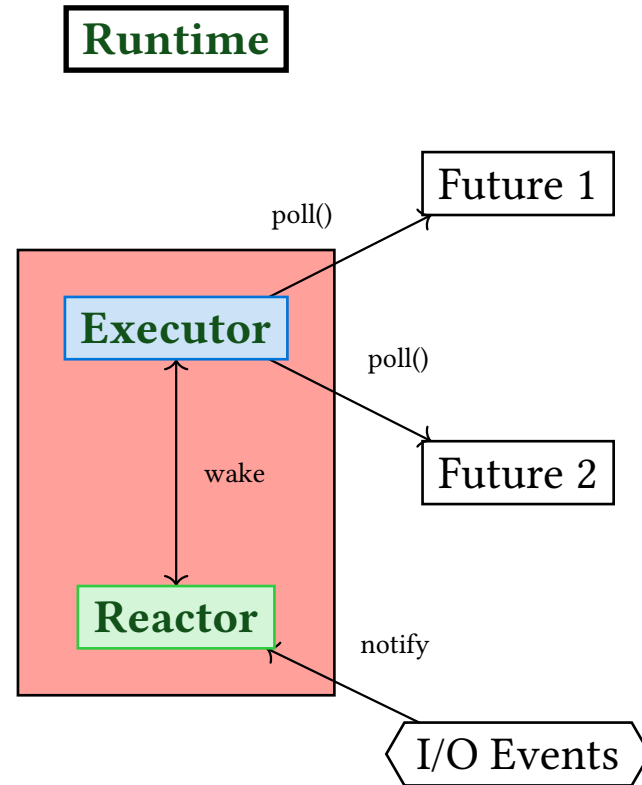
- **reactor**: reacting to IO events asynchronously
- **executor**: pushing futures forward concurrently

Rust does not have a “built-in” runtime, but several options are available:

- futures offers a bare bones executor
ThreadPool
- tokio offers a Runtime with reactor included
(supplies “time” etc.)

Warning

Embassy is not a traditional async run-time, built for small embedded devices. It has no allocator (no heap).



```
1  use tokio::time;
2  async fn count_to(count: i32) {
3      for i in 0..count {
4          println!("Count in task: {i}!");
5          time::sleep(time::Duration::from_millis(5)).await;
6      }
7  }
8  #[tokio::main] // Optional macro to start async multi-threaded runtime.
9  async fn main() {
10     tokio::spawn(count_to(10));
11     for i in 0..5 {
12         println!("Main task: {i}");
13         time::sleep(time::Duration::from_millis(5)).await;
14     }
15 }
```

[\(playground link\)](#)

Async tasks are also aborted if not awaited in main.

5.3. Tasks

A task is a top-level future spawned into the runtime.

5.3. Tasks

A task is a top-level future spawned into the runtime.

Warning

In multi-threaded runtimes, tasks must be `Send + 'static` and may be moved between worker threads by the work-stealing scheduler.

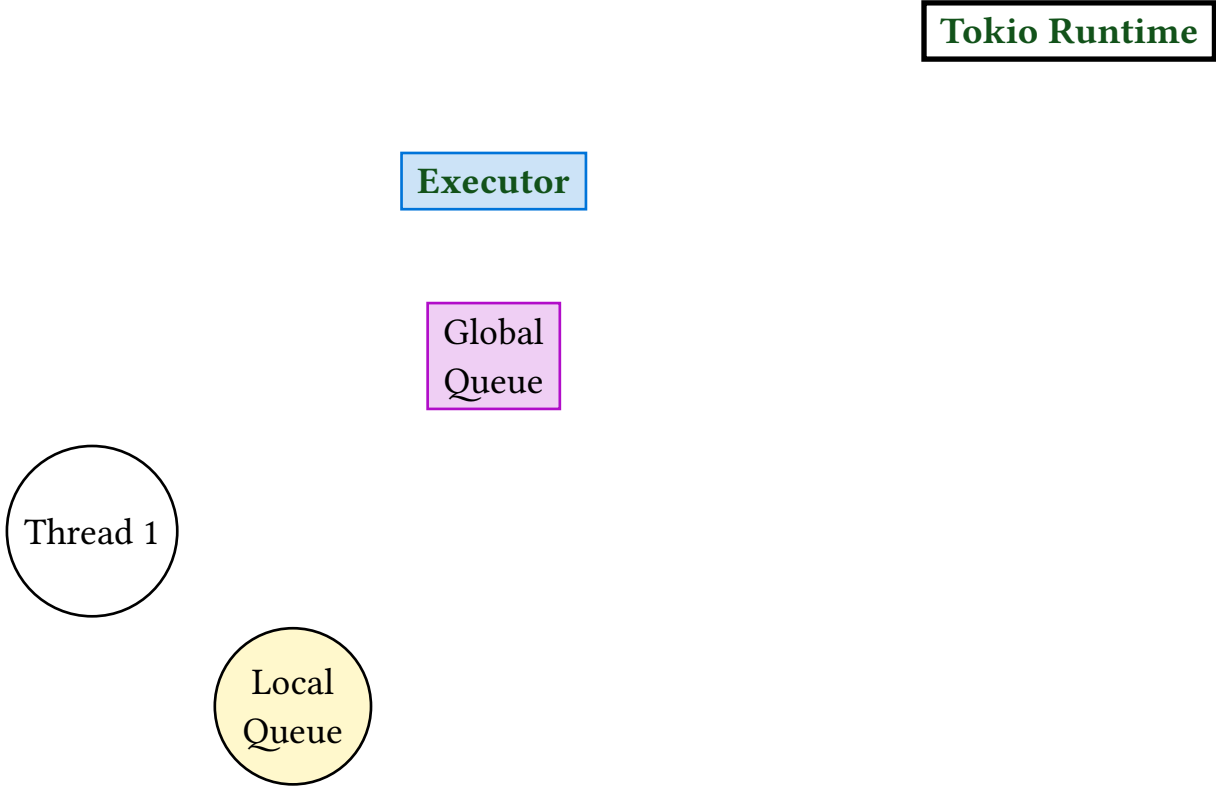
```
1  async fn main() -> io::Result<()> {  
2      let listener = TcpListener::bind("127.0.0.1:0").await?;  
3  
4      loop {  
5          let (mut socket, addr) = listener.accept().await?;  
6  
7          tokio::spawn(async move {  
8              // Handle connection  
9          });  
10     }  
11 }
```

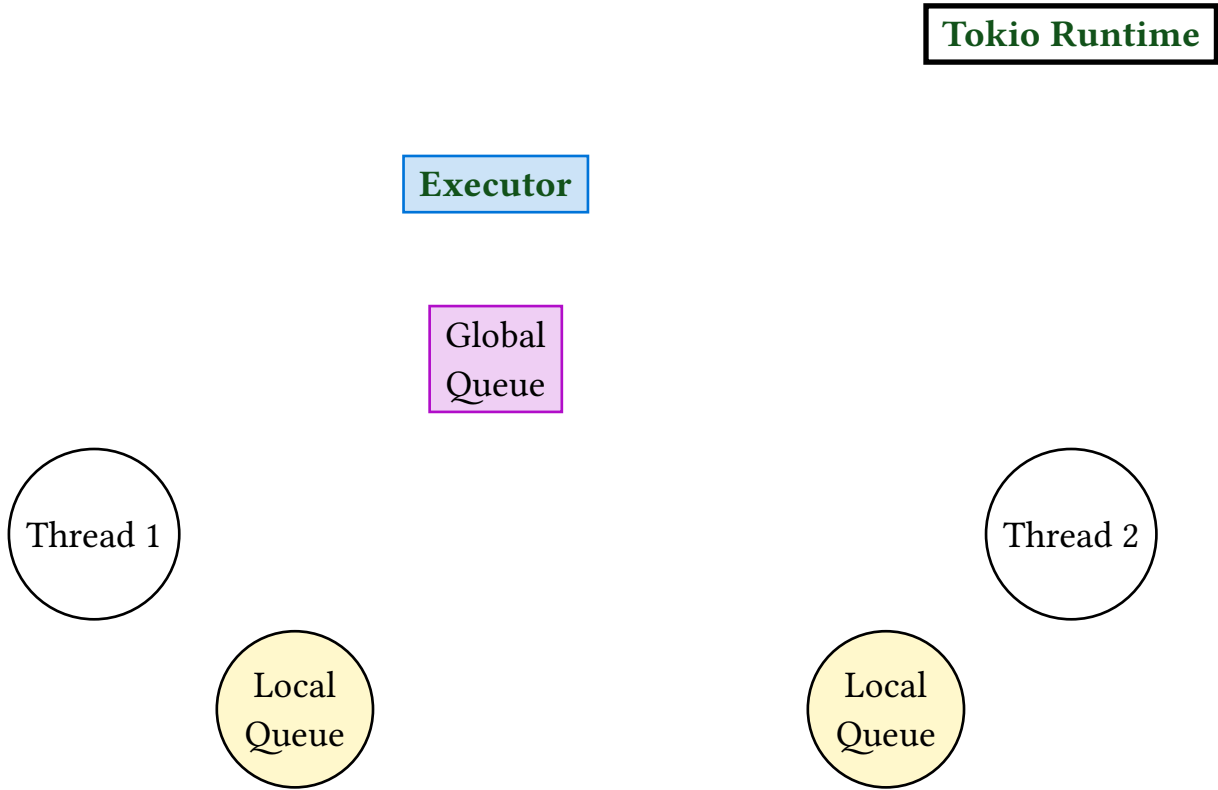
[\(playground link\)](#)

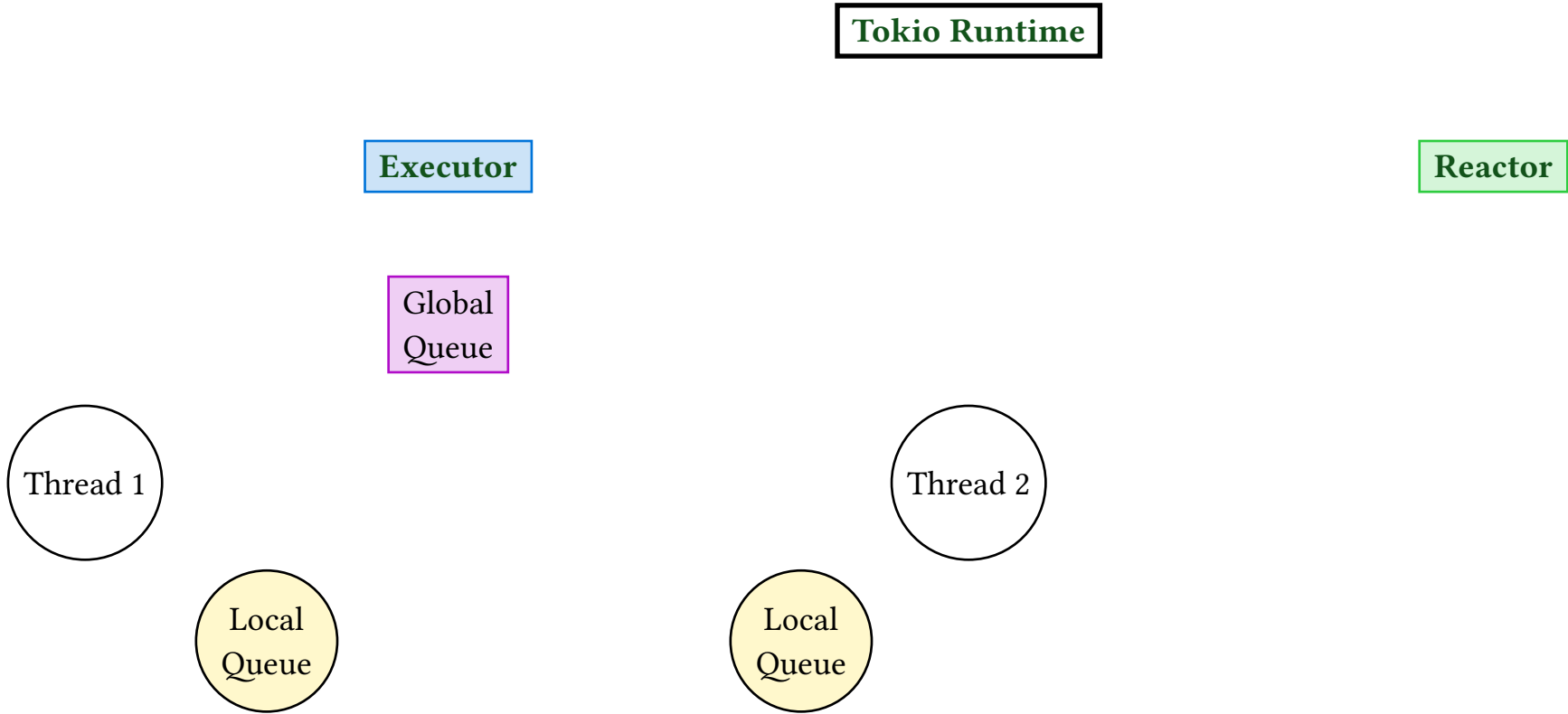
Tokio Runtime

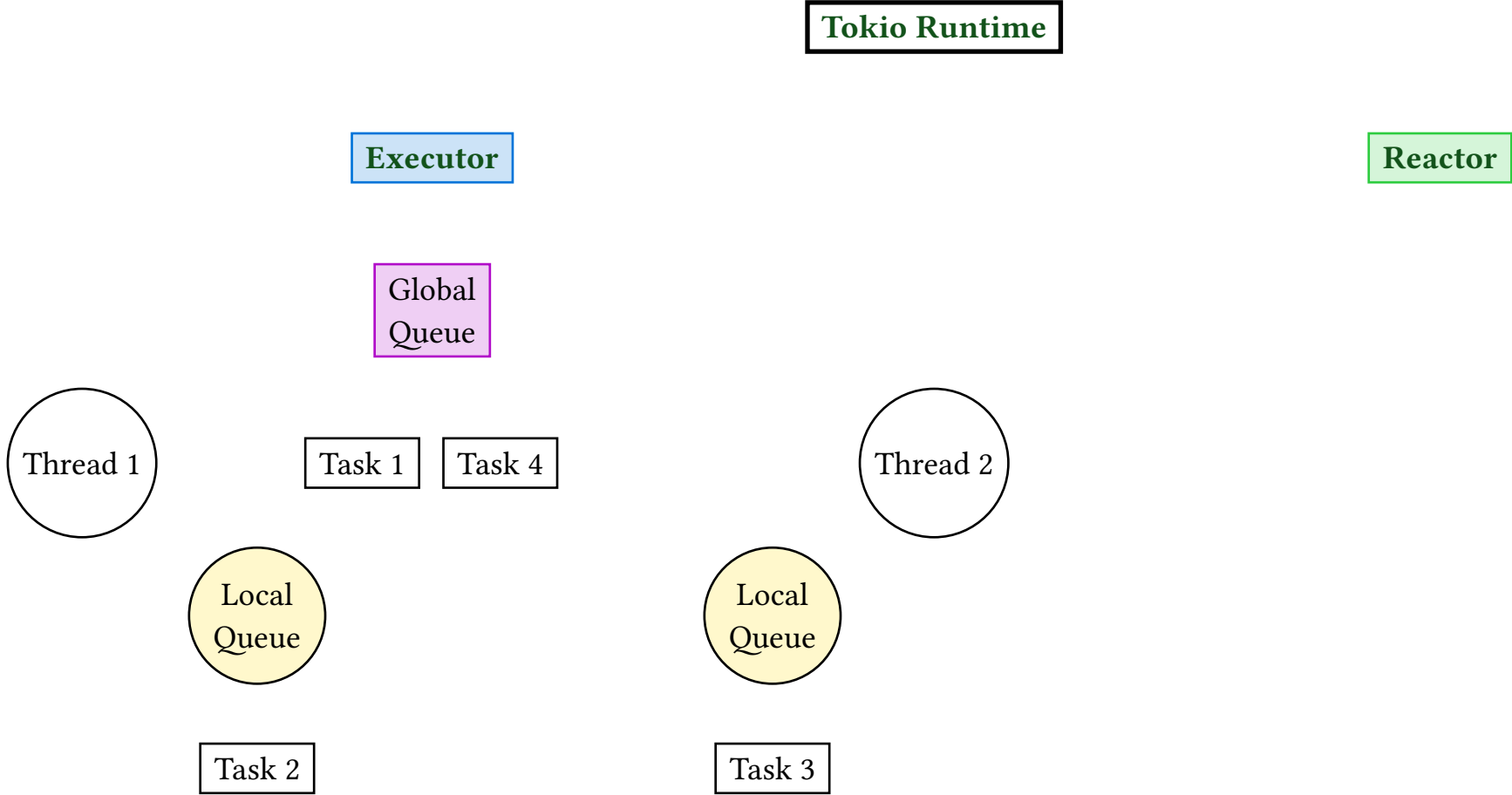


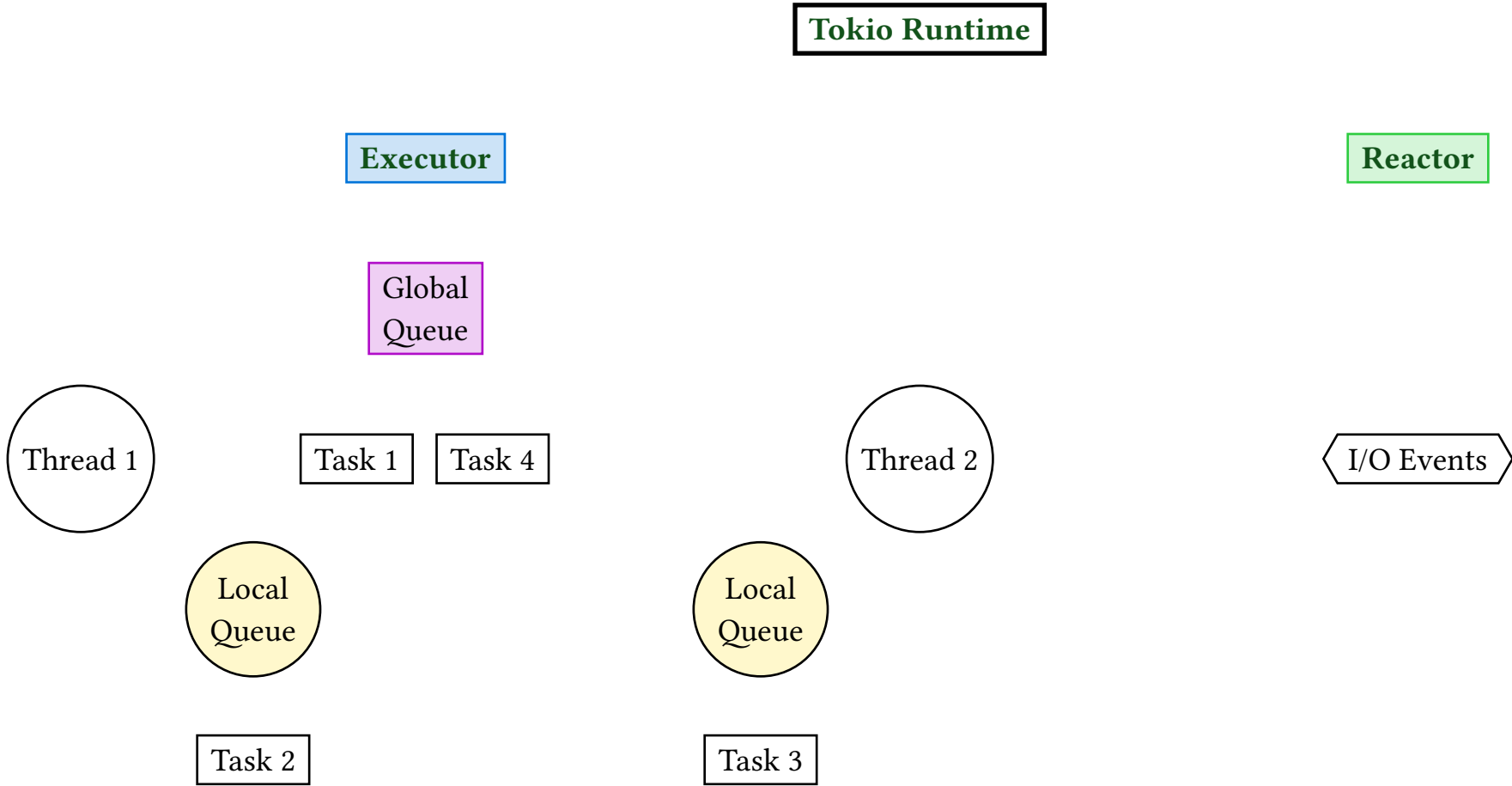


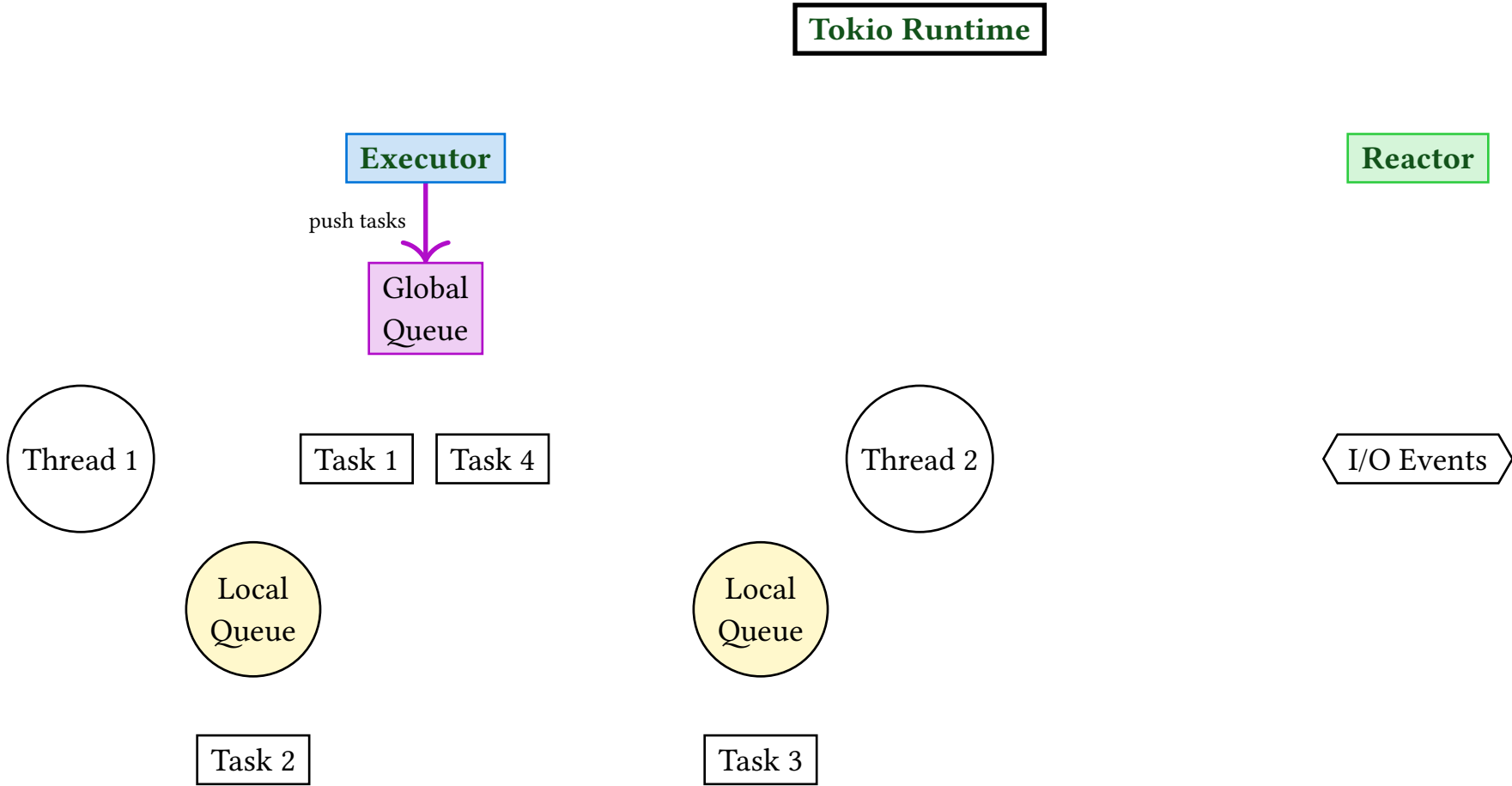




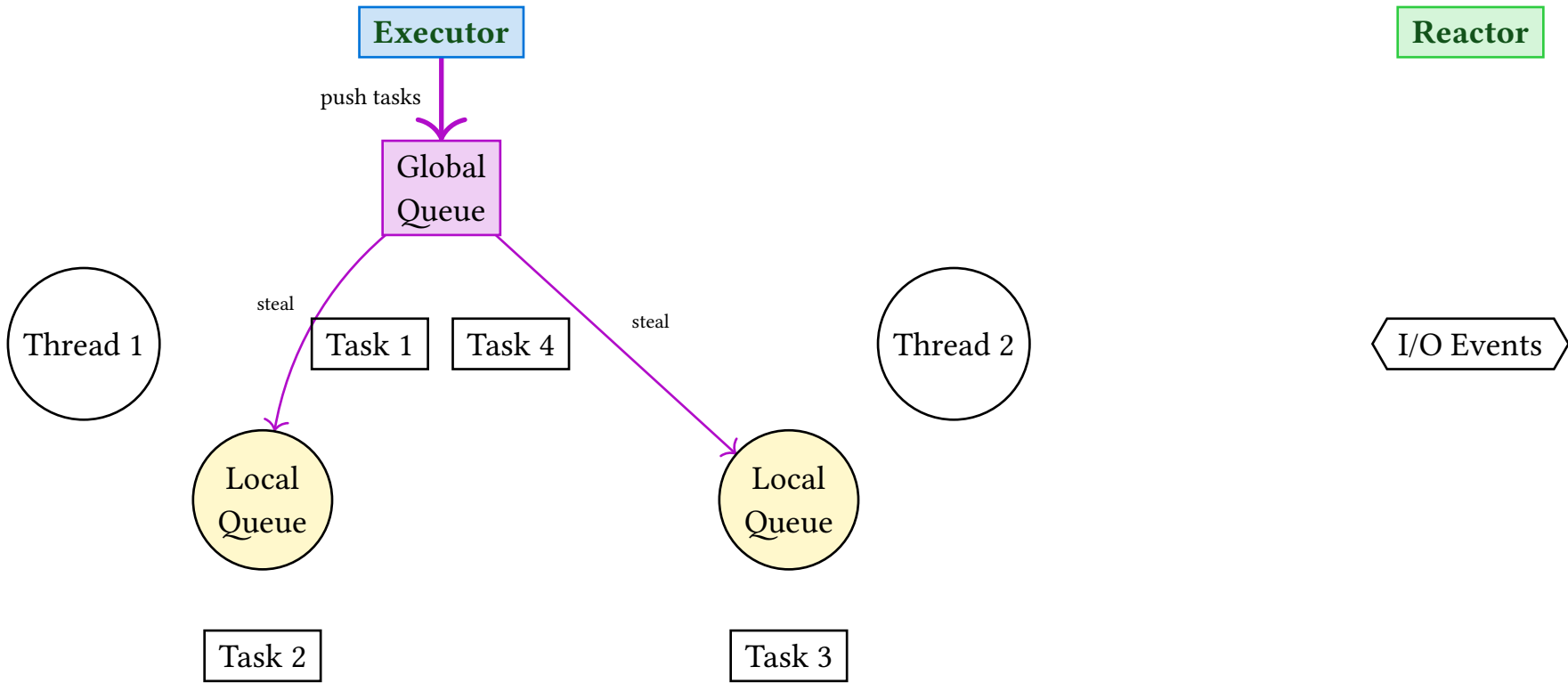




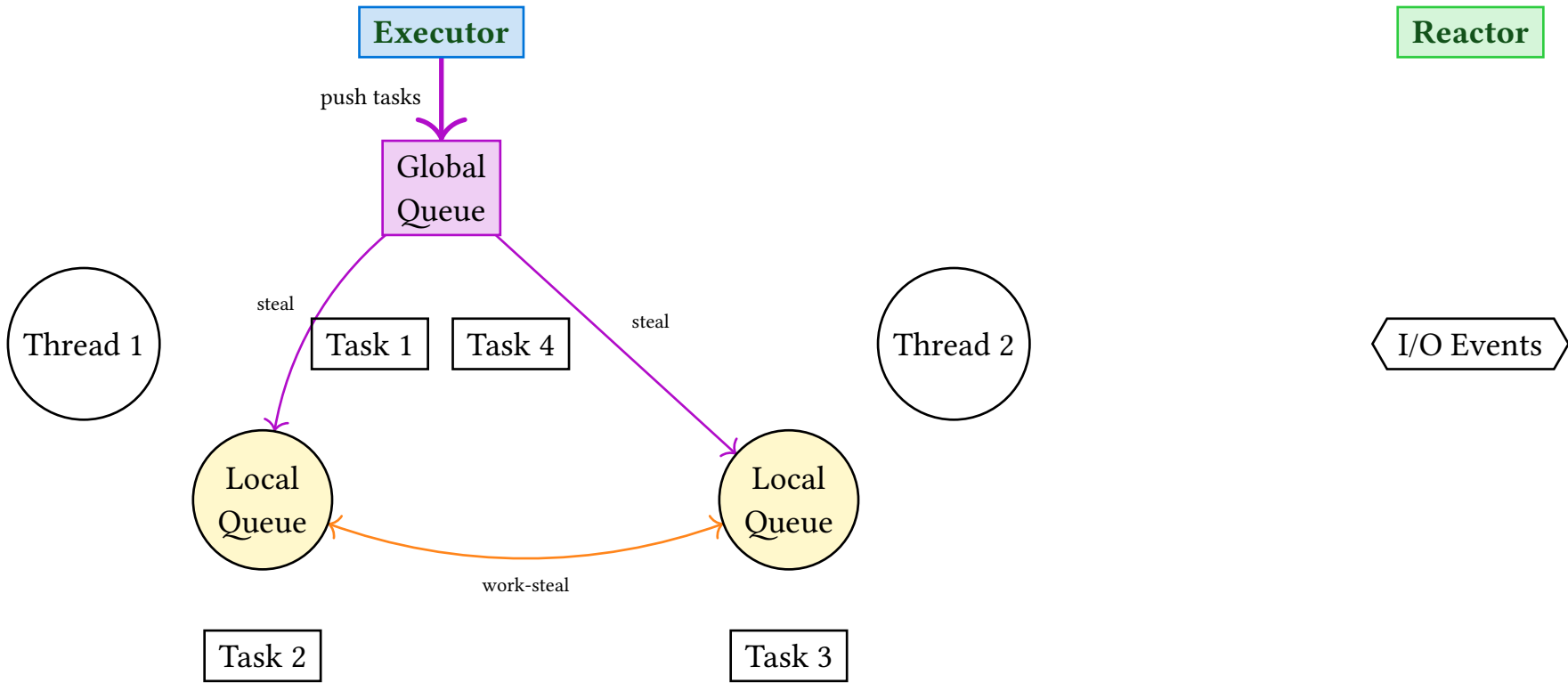




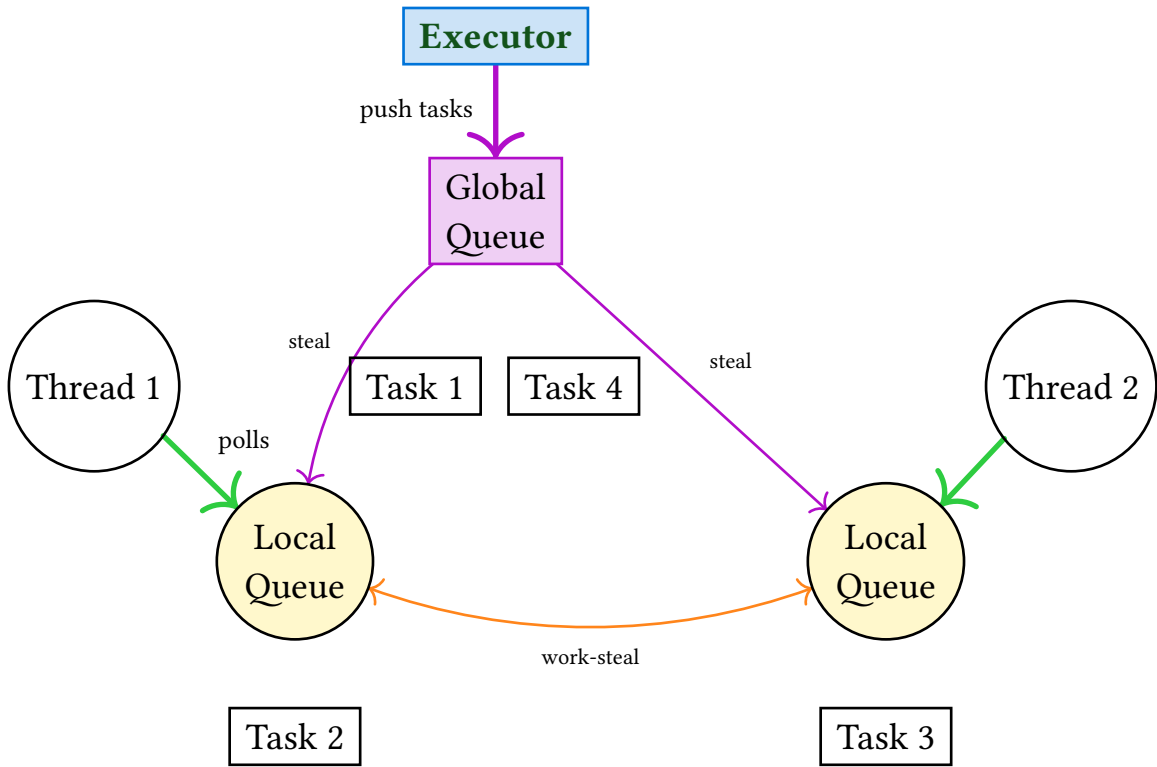
Tokio Runtime



Tokio Runtime



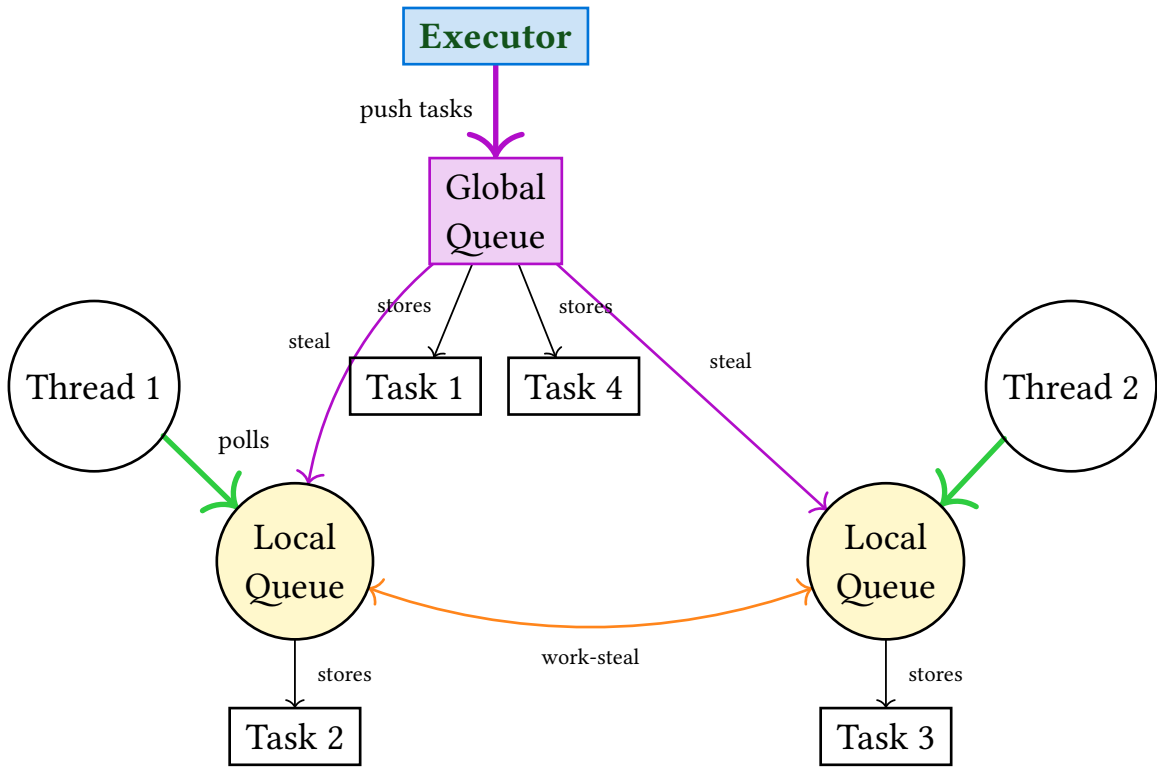
Tokio Runtime



Reactor

I/O Events

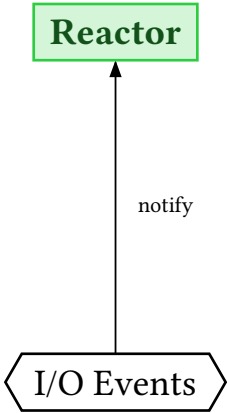
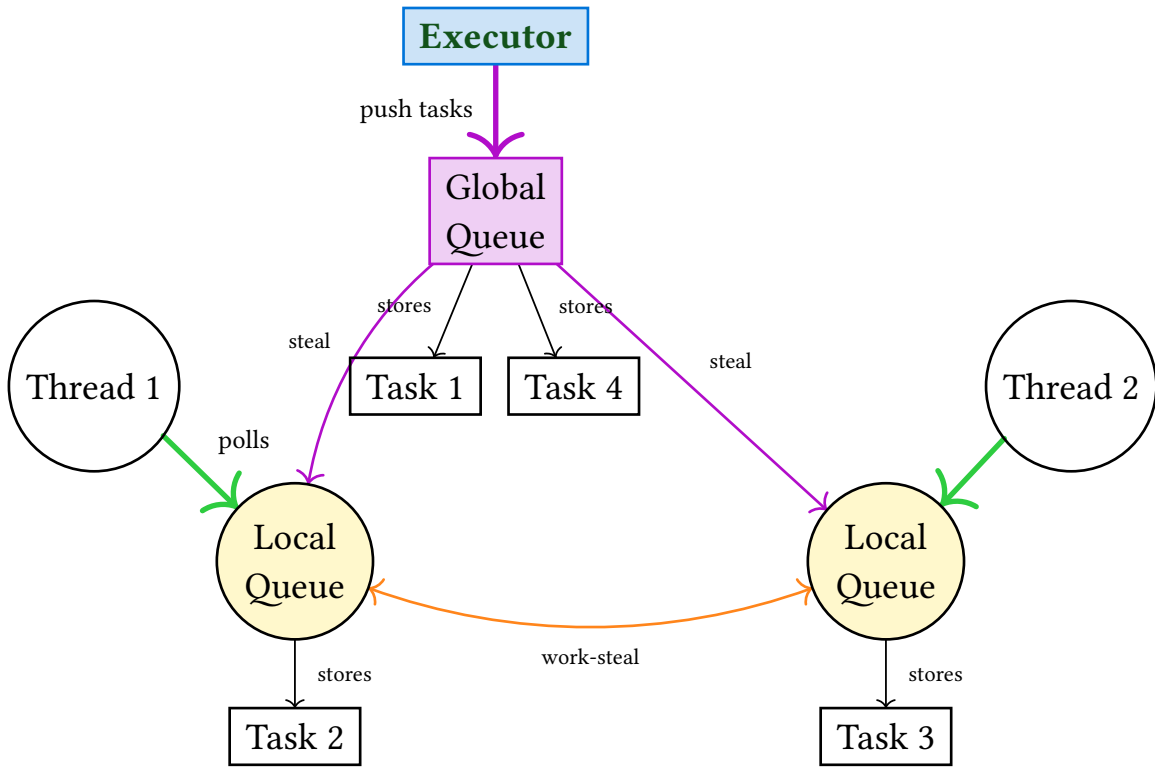
Tokio Runtime

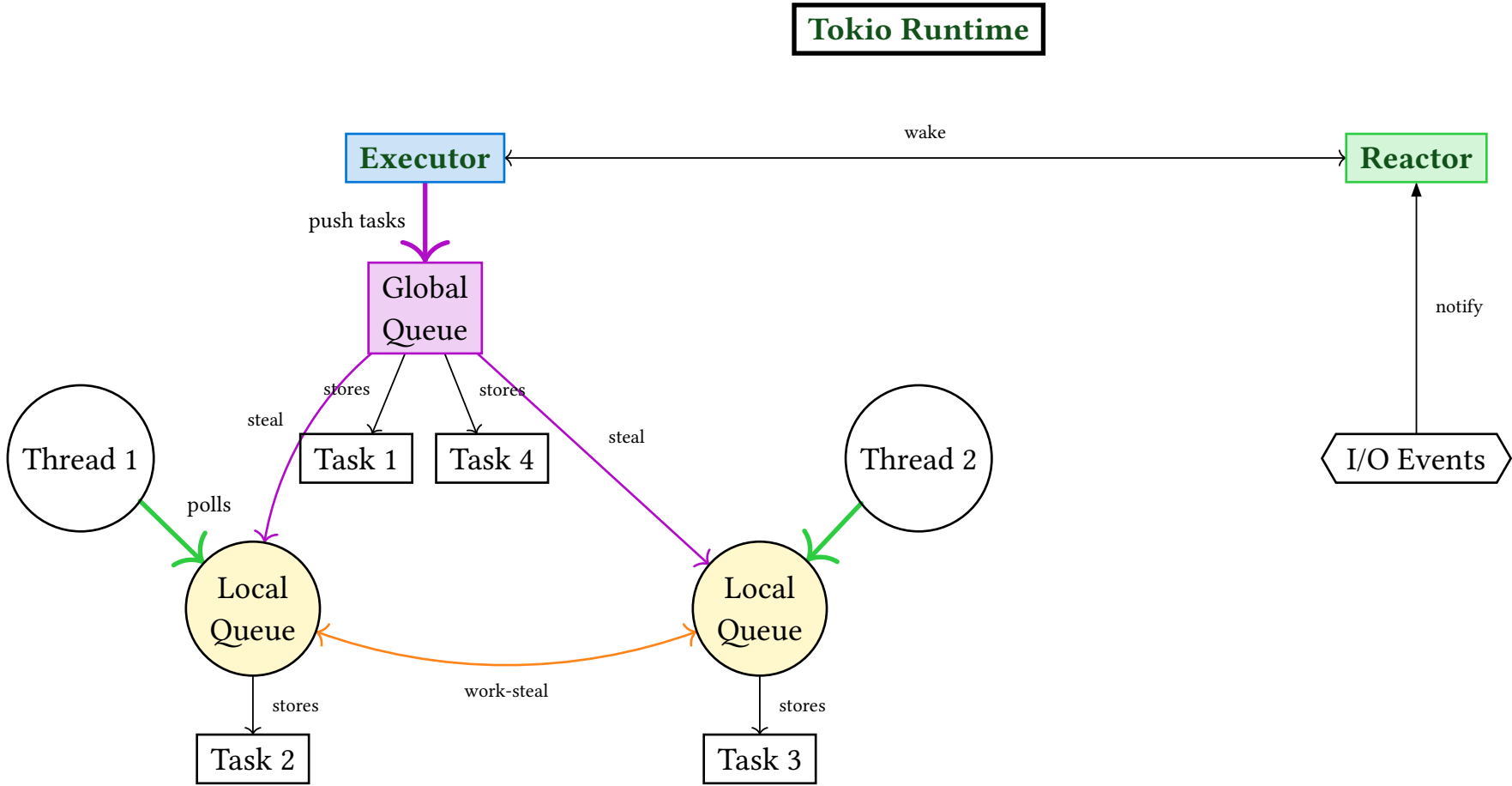


Reactor

I/O Events

Tokio Runtime





Info

An async block like `async move {}` is like a scope block that may contain awaits.

Refactor `session7/examples/tasks.rs`:

- Create async helper function from async block
- Improve error handling

1.	Student presentations	1
2.	Asynchronous programming	4
3.	Asynchronous Rust	8
4.	Rust's async implementation	15
5.	Ecosystem	22
6.	Common async datatypes and functions	28
6.1.	Channels	29
6.2.	Joining two futures	30
6.3.	Joining an iterable	32
6.4.	Select	34
6.5.	Select exercise	36
6.6.	Async Mutex	37
7.	Streams	38
8.	Pitfalls	44
9.	Exercise: async philosophers	49
10.	Exercise: broadcast chat application	53

```
1 async fn ping_handler(mut input: mpsc::Receiver<()>) {  
2     let mut count: usize = 0;  
3     while let Some(_) = input.recv().await {  
4         count += 1;  
5         println!("Received {count} pings so far.");  
6     }  
7 }
```

[\(playground link\)](#)

```
1 async fn ping_handler(mut input: mpsc::Receiver<()>) {
2     let mut count: usize = 0;
3     while let Some(_) = input.recv().await {
4         count += 1;
5         println!("Received {count} pings so far.");
6     }
7 }
```

[\(playground link\)](#)

```
1 async fn main() {
2     let (sender, receiver) = mpsc::channel(32);
3     let handler_task = tokio::spawn(ping_handler(receiver));
4     for i in 0..10 {
5         sender.send(()).await.expect("Failed to send");
6     }
7     drop(sender);
8     handler_task.await.expect("Handler failed");
9 }
```

[\(playground link\)](#)

6.2. Joining two futures

When you need to wait for exactly two futures concurrently, use the `join!` macro:

```
1  use tokio::join;
2
3  async fn fetch_user(id: u32) -> User { /* ... */ }
4  async fn fetch_posts(user_id: u32) -> Vec<Post> { /* ... */ }
5
6  #[tokio::main]
7  async fn main() {
8      let (user, posts) = join!(
9          fetch_user(1),
10         fetch_posts(1)
11     );
12     println!("User: {user:?}, Posts: {}", posts.len());
13 }
```

[\(playground link\)](#)

6.2. Joining two futures

When you need to wait for exactly two futures concurrently, use the `join!` macro:

```
1 use tokio::join;
2
3 async fn fetch_user(id: u32) -> User { /* ... */ }
4 async fn fetch_posts(user_id: u32) -> Vec<Post> { /* ... */ }
5
6 #[tokio::main]
7 async fn main() {
8     let (user, posts) = join!(
9         fetch_user(1),
10        fetch_posts(1)
11    );
12    println!("User: {user:?}, Posts: {}", posts.len());
13 }
```

[\(playground link\)](#)

Unlike sequential `await`s, both futures run concurrently. If one fails, both are awaited before returning the error.

6.2. Joining two futures

Exercise: Refactor `session7/examples/join.rs` to use `join!` instead of sequential awaits.

6.3. Joining an iterable

We might need to do the same async function multiple times:

```
1 async fn size_of_page(url: &str) -> Result<usize> {  
2     let resp = request::get(url).await?;  
3     Ok(resp.text().await?.len())  
4 }
```

[\(playground link\)](#)

How can we await a list of futures concurrently?

6.3. Joining an iterable

We might need to do the same async function multiple times:

```
1 async fn size_of_page(url: &str) -> Result<usize> {  
2     let resp = reqwest::get(url).await?;  
3     Ok(resp.text().await?.len())  
4 }
```

[\(playground link\)](#)

How can we await a list of futures concurrently?

We use the `join_all` function.

6.3. Joining an iterable

We might need to do the same async function multiple times:

```
1 async fn size_of_page(url: &str) -> Result<usize> {  
2     let resp = request::get(url).await?;  
3     Ok(resp.text().await?.len())  
4 }
```

[\(playground link\)](#)

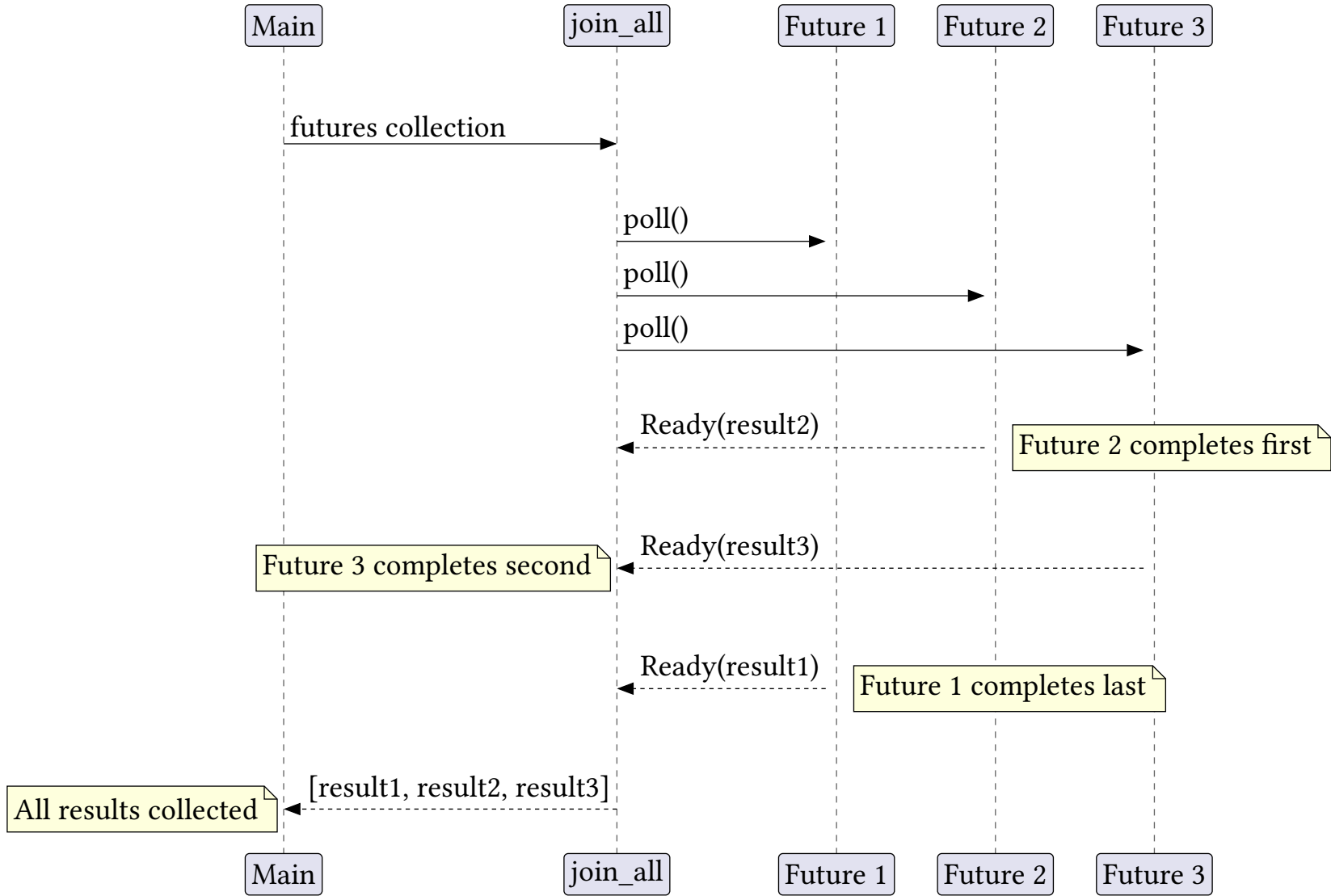
How can we await a list of futures concurrently?

We use the `join_all` function.

```
1 async fn main() {  
2     let urls = ["https://google.com", "https://httpbin.org/ip", /*...*/];  
3     let futures = urls.into_iter().map(size_of_page);  
4     let results = future::join_all(futures).await;  
5     println!("{results:?}");  
6 }
```

[\(playground link\)](#)

6.3. Joining an iterable

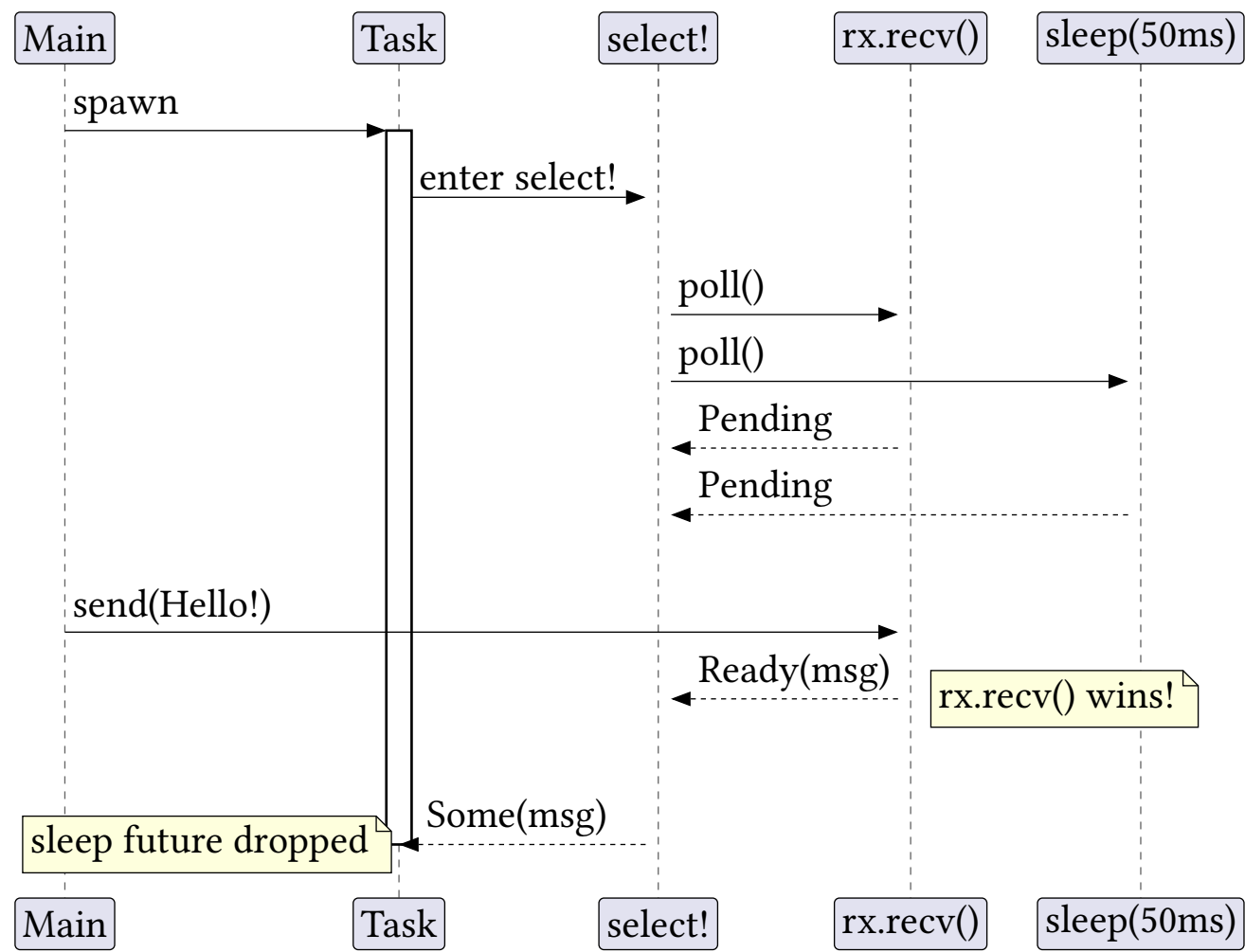


6.4. Select

Waits until any of a set of futures is ready. Similar to `Promise.race` or Python's `asyncio.wait()`.

```
1  #[tokio::main]
2  async fn main() {
3      let (tx, mut rx) = mpsc::channel(32);
4
5      tokio::spawn(async move {
6          tokio::select! {
7              Some(msg) = rx.recv() => println!("got: {msg}"),
8              _ = sleep(Duration::from_millis(50)) => println!("timeout"),
9          };
10     });
11
12     sleep(Duration::from_millis(10)).await;
13     tx.send(String::from("Hello!")).await?;
14     // ...
15 }
```

([playground link](#))



6.5. Select exercise

6. Common async datatypes and functions

Use `tokio::select!` to implement a timeout mechanism for a long-running operation.

```
1  async fn fetch_data() -> String {
2      sleep(Duration::from_secs(5)).await;
3      "Data fetched".to_string()
4  }
5
6  #[tokio::main]
7  async fn main() {
8      // TODO: Use select! to race fetch_data() against a 2-second timeout
9      // Print "Success: {data}" if fetch completes
10     // Print "Timeout!" if the timeout occurs first
11 }
```

([playground link](#))

6.5. Select exercise

6. Common async datatypes and functions

Use `tokio::select!` to implement a timeout mechanism for a long-running operation.

```
1  async fn fetch_data() -> String {
2      sleep(Duration::from_secs(5)).await;
3      "Data fetched".to_string()
4  }
5
6  #[tokio::main]
7  async fn main() {
8      // TODO: Use select! to race fetch_data() against a 2-second timeout
9      // Print "Success: {data}" if fetch completes
10     // Print "Timeout!" if the timeout occurs first
11 }
```

([playground link](#))

Exercise: Complete `session7/examples/select.rs` to implement timeout logic using `tokio::select!`.

Prefer `std::sync::Mutex` by default - it's faster and cheaper for short critical sections.

6.6. Async Mutex

Prefer `std::sync::Mutex` by default - it's faster and cheaper for short critical sections.

Use `tokio::sync::Mutex` only when you need to hold the lock across `.await` points:

```
1 // Won't compile: std::sync::MutexGuard is not Send
2 let guard = std_mutex.lock().unwrap();
3 some_async_operation().await; // Error!
4 drop(guard); \(playground link\)
```

```
1 // Works: tokio::sync::MutexGuard is Send
2 let guard = tokio_mutex.lock().await;
3 some_async_operation().await; // OK
4 drop(guard); \(playground link\)
```

6.6. Async Mutex

Prefer `std::sync::Mutex` by default - it's faster and cheaper for short critical sections.

Use `tokio::sync::Mutex` only when you need to hold the lock across `.await` points:

```
1 // Won't compile: std::sync::MutexGuard is not Send
2 let guard = std_mutex.lock().unwrap();
3 some_async_operation().await; // Error!
4 drop(guard);
```

[\(playground link\)](#)

```
1 // Works: tokio::sync::MutexGuard is Send
2 let guard = tokio_mutex.lock().await;
3 some_async_operation().await; // OK
4 drop(guard);
```

[\(playground link\)](#)

The `Send` requirement exists because futures may move between threads at `await` points.

`std::sync::MutexGuard` is deliberately not `Send` to prevent deadlocks.

6.6. Async Mutex

Prefer `std::sync::Mutex` by default - it's faster and cheaper for short critical sections.

Use `tokio::sync::Mutex` only when you need to hold the lock across `.await` points:

```
1 // Won't compile: std::sync::MutexGuard is not Send
2 let guard = std_mutex.lock().unwrap();
3 some_async_operation().await; // Error!
4 drop(guard);
```

[\(playground link\)](#)

```
1 // Works: tokio::sync::MutexGuard is Send
2 let guard = tokio_mutex.lock().await;
3 some_async_operation().await; // OK
4 drop(guard);
```

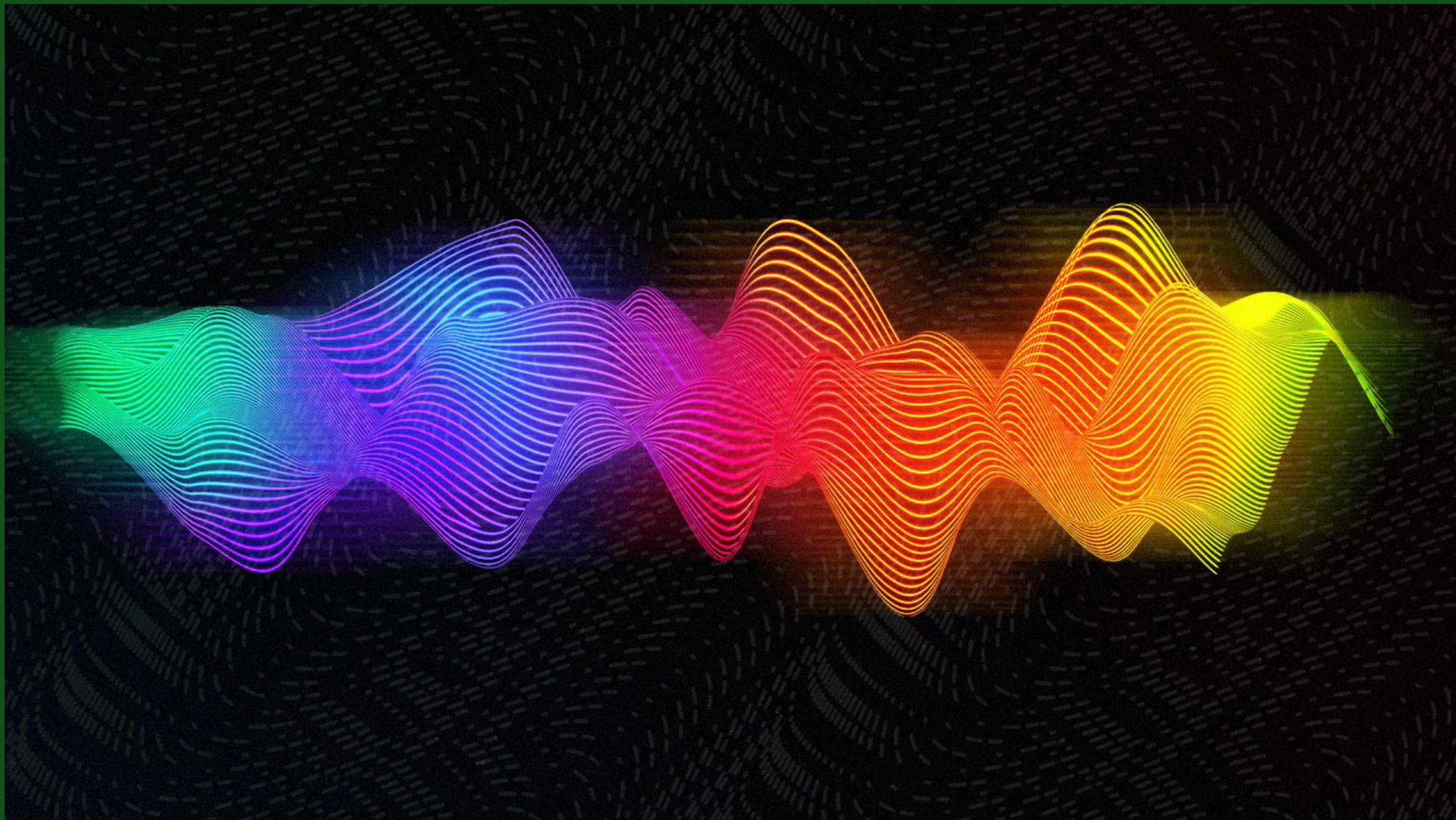
[\(playground link\)](#)

The `Send` requirement exists because futures may move between threads at `await` points.

`std::sync::MutexGuard` is deliberately not `Send` to prevent deadlocks.

Alternative: Consider using message passing (channels) instead of shared mutable state.

1.	Student presentations	1
2.	Asynchronous programming	4
3.	Asynchronous Rust	8
4.	Rust's async implementation	15
5.	Ecosystem	22
6.	Common async datatypes and functions	28
7.	Streams	38
7.1.	Like iterators	39
7.2.	Awaiting many futures	40
7.3.	Stream operators	41
7.4.	Stream exercise	42
8.	Pitfalls	44
9.	Exercise: async philosophers	49
10.	Exercise: broadcast chat application	53



7.1. Like iterators

Streams are runtime-agnostic asynchronous iterators:

```
1  use futures::stream::{self, StreamExt};
2
3  let stream = stream::iter(vec!['a', 'b', 'c']);
4
5  let mut stream = stream.enumerate();
6
7  assert_eq!(stream.next().await, Some((0, 'a')));
8  assert_eq!(stream.next().await, Some((1, 'b')));
9  assert_eq!(stream.next().await, Some((2, 'c')));
10 assert_eq!(stream.next().await, None);
```

[\(playground link\)](#)

7.2. Awaiting many futures

The `FuturesUnordered` is a `Stream` that yields results as futures complete (in completion order, not insertion order).

7.2. Awaiting many futures

The `FuturesUnordered` is a `Stream` that yields results as futures complete (in completion order, not insertion order).

```
1  async fn process_item(id: u32) -> String {
2      sleep(Duration::from_millis(100 * id as u64)).await;
3      format!("Processed item {id}")
4  }
5
6  async fn main() {
7      let mut futures = FuturesUnordered::new();
8
9      for i in 1..=5 { futures.push(process_item(i)); }
10
11     while let Some(result) = futures.next().await {
12         println!("{result}");
13     }
14 }
```

[\(playground link\)](#)

7.3. Stream operators

You can also apply “adapters” to streams (similar to iterator adapters):

```
1  #[tokio::main]
2  async fn main() {
3      let stream = stream::iter(1..=10);
4
5      let result: Vec<_> = stream
6          .filter(|x| futures::future::ready(x % 2 == 0))
7          .map(|x| x * 2)
8          .collect()
9          .await;
10
11     println!("{result:?}"); // [4, 8, 12, 16, 20]
12 }
```

[\(playground link\)](#)

7.3. Stream operators

You can also apply “adapters” to streams (similar to iterator adapters):

```
1  #[tokio::main]
2  async fn main() {
3      let stream = stream::iter(1..=10);
4
5      let result: Vec<_> = stream
6          .filter(|x| futures::future::ready(x % 2 == 0))
7          .map(|x| x * 2)
8          .collect()
9          .await;
10
11     println!("{result:?}"); // [4, 8, 12, 16, 20]
12 }
```

([playground link](#))

Common adapters: map, filter, filter_map, fold, take, skip, zip, chain

See: Futures StreamExt trait and my project <https://github.com/wvhulle/clone-stream>

7.4. Stream exercise

Process a stream of numbers using adapters to:

1. Filter out numbers less than 5
2. Square each number
3. Take only the first 3 results

```
1  use futures::future::ready;
2  use futures::stream::{self, StreamExt};
3
4  #[tokio::main]
5  async fn main() {
6      let numbers = stream::iter(1..=20);
7
8      // TODO: Chain filter, map, and take adapters
9      // to get the first 3 squares of numbers >= 5
10     // Use ready() in the filter predicate
11
12     let result: Vec<_> = numbers.collect().await;
13     println!("{result:?}");
14 }
```

[\(playground link\)](#)

7.4. Stream exercise

Process a stream of numbers using adapters to:

1. Filter out numbers less than 5
2. Square each number
3. Take only the first 3 results

```
1  use futures::future::ready;
2  use futures::stream::{self, StreamExt};
3
4  #[tokio::main]
5  async fn main() {
6      let numbers = stream::iter(1..=20);
7
8      // TODO: Chain filter, map, and take adapters
9      // to get the first 3 squares of numbers >= 5
10     // Use ready() in the filter predicate
11
12     let result: Vec<_> = numbers.collect().await;
13     println!("{result:?}");
14 }
```

([playground link](#))

Exercise: Complete `session7/examples/streams.rs` to transform the stream using adapters.

1.	Student presentations	1
2.	Asynchronous programming	4
3.	Asynchronous Rust	8
4.	Rust's async implementation	15
5.	Ecosystem	22
6.	Common async datatypes and functions	28
7.	Streams	38
8.	Pitfalls	44
8.1.	Blocking operations prevent context switching	45
8.2.	Async for limited parallelism	46
8.3.	Pinning too much	47
8.4.	Forget handling deadlocks	48
9.	Exercise: async philosophers	49
10.	Exercise: broadcast chat application	53

8.1. Blocking operations prevent context switching

```
1  async fn sleep_ms(start: &Instant, id: u64, duration_ms: u64) {
2      std::thread::sleep(std::time::Duration::from_millis(duration_ms));
3      println!("future {id} finished after {}ms", start.elapsed().as_millis());
4  }
5
6  #[tokio::main(flavor = "current_thread")]
7  async fn main() {
8      let start = Instant::now();
9      let futures = (1..=10).map(|t| sleep_ms(&start, t, t * 10));
10     join_all(futures).await;
11 }
```

[\(playground link\)](#)

8.1. Blocking operations prevent context switching

```
1  async fn sleep_ms(start: &Instant, id: u64, duration_ms: u64) {
2      std::thread::sleep(std::time::Duration::from_millis(duration_ms));
3      println!("future {id} finished after {}ms", start.elapsed().as_millis());
4  }
5
6  #[tokio::main(flavor = "current_thread")]
7  async fn main() {
8      let start = Instant::now();
9      let futures = (1..=10).map(|t| sleep_ms(&start, t, t * 10));
10     join_all(futures).await;
11 }
```

[\(playground link\)](#)

While `std::thread::sleep` blocks, the executor cannot poll other tasks. How to run them concurrently?

8.1. Blocking operations prevent context switching

```
1  async fn sleep_ms(start: &Instant, id: u64, duration_ms: u64) {
2      std::thread::sleep(std::time::Duration::from_millis(duration_ms));
3      println!("future {id} finished after {}ms", start.elapsed().as_millis());
4  }
5
6  #[tokio::main(flavor = "current_thread")]
7  async fn main() {
8      let start = Instant::now();
9      let futures = (1..=10).map(|t| sleep_ms(&start, t, t * 10));
10     join_all(futures).await;
11 }
```

[\(playground link\)](#)

While `std::thread::sleep` blocks, the executor cannot poll other tasks. How to run them concurrently?

Use async-aware operations like `tokio::time::sleep` that yield control back to the executor, allowing context switching.

8.2. Async for limited parallelism

Warning

Async tasks run on a limited thread pool and don't provide true parallelism for CPU-bound work.

Async is designed for I/O concurrency, not CPU parallelism:

- Spawning 10,000 async tasks doesn't create 10,000 workers
- Tasks execute sequentially on the available thread pool
- CPU-bound work sees no benefit from async spawning

8.2. Async for limited parallelism

Warning

Async tasks run on a limited thread pool and don't provide true parallelism for CPU-bound work.

Async is designed for I/O concurrency, not CPU parallelism:

- Spawning 10,000 async tasks doesn't create 10,000 workers
- Tasks execute sequentially on the available thread pool
- CPU-bound work sees no benefit from async spawning

Recommendation: If there's no real I/O waiting:

- Use a small number of OS threads instead (`std::thread` or `rayon`)
- Match thread count to CPU cores for optimal performance
- Reserve async for coordinating I/O operations

8.3. Pinning too much

The compiler may tell you to use `Box::pin` to pin futures to the heap.

Heap pinning (avoid in hot paths):

```
1 loop {
2     let fut = Box::pin(async {
3         process().await
4     });
5     tokio::select! {
6         r = fut => handle(r),
7         _ = shutdown() => break,
8     }
9 }
```

[\(playground link\)](#)

Stack pinning (zero-cost):

```
1 use tokio::pin;
2 loop {
3     let fut = async {
4         process().await
5     };
6     pin!(fut);
7     tokio::select! {
8         r = fut => handle(r),
9         _ = shutdown() => break,
10    }
11 }
```

[\(playground link\)](#)

Use `Box::pin` for collections or return values, `pin!` for hot paths, `pin_project` for custom structs.

8.4. Forget handling deadlocks

Asynchronous code is still susceptible to deadlocks.

A deadlock occurs when four conditions are met simultaneously (Coffman conditions):

Condition	Description
Mutual exclusion	Resources cannot be shared (only one thread at a time)
Hold and wait	Threads hold resources while waiting for others
No preemption	Resources cannot be forcibly taken from threads
Circular wait	Threads form a cycle waiting for each other's resources

8.4. Forget handling deadlocks

Asynchronous code is still susceptible to deadlocks.

A deadlock occurs when four conditions are met simultaneously (Coffman conditions):

Condition	Description
Mutual exclusion	Resources cannot be shared (only one thread at a time)
Hold and wait	Threads hold resources while waiting for others
No preemption	Resources cannot be forcibly taken from threads
Circular wait	Threads form a cycle waiting for each other's resources

To prevent deadlock, break at least one condition:

- **Lock ordering:** Always acquire locks in the same order (breaks circular wait)
- **Timeout:** Release locks if waiting too long (breaks hold and wait)
- **Try-lock:** Use `try_lock()` instead of `lock()` (breaks hold and wait)

1.	Student presentations	1
2.	Asynchronous programming	4
3.	Asynchronous Rust	8
4.	Rust's async implementation	15
5.	Ecosystem	22
6.	Common async datatypes and functions	28
7.	Streams	38
8.	Pitfalls	44
9.	Exercise: async philosophers	49
9.1.	Context	50
9.2.	Assignment	51
9.3.	Additional bonus	52
10.	Exercise: broadcast chat application	53

9.1. Context

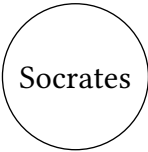
Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

Chopstick 2

Chopstick 0

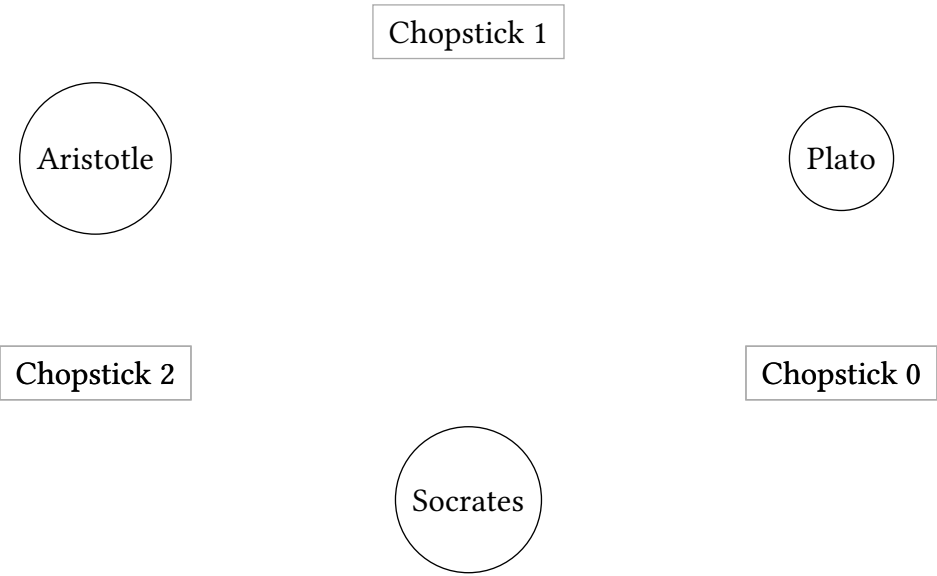


9.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

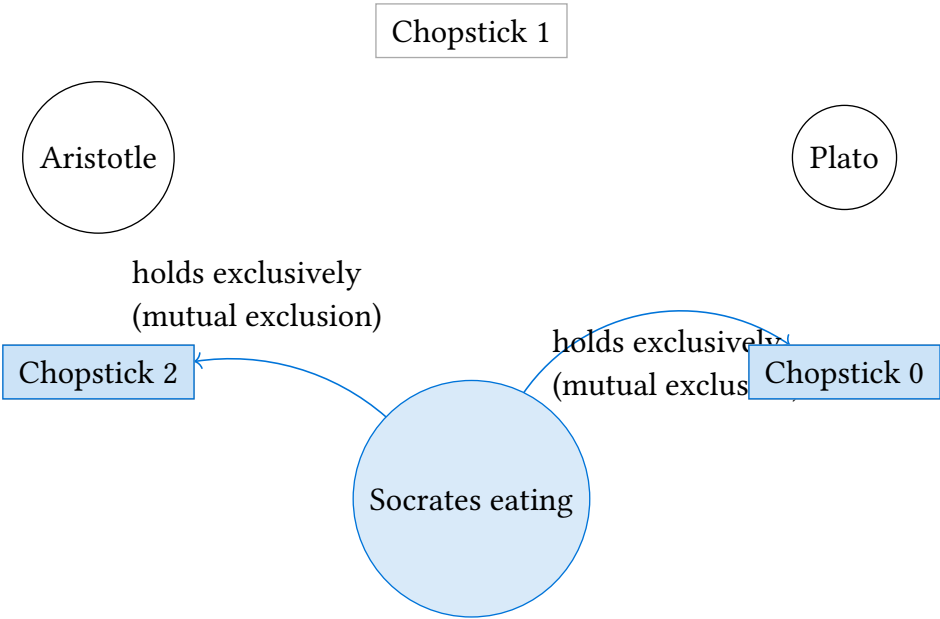


9.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

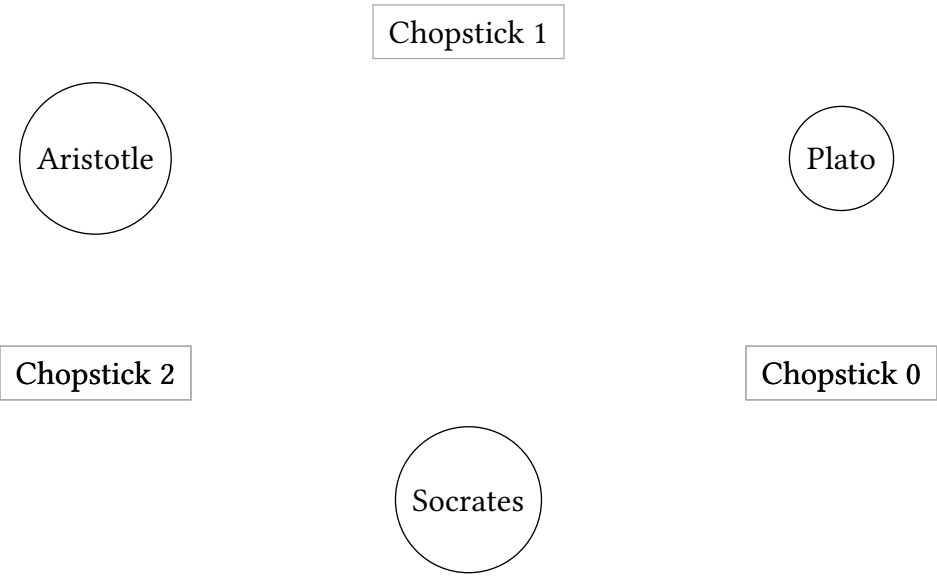


9.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

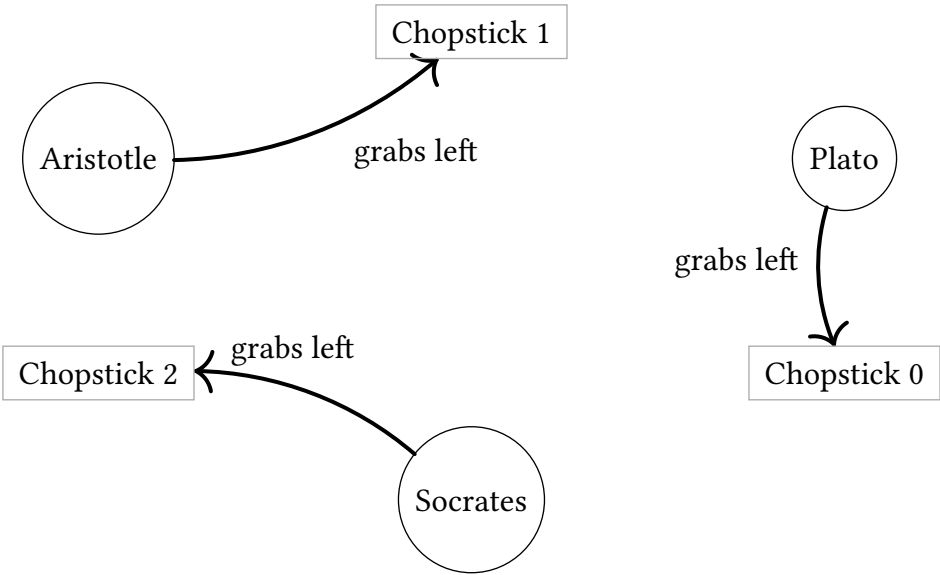


9.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

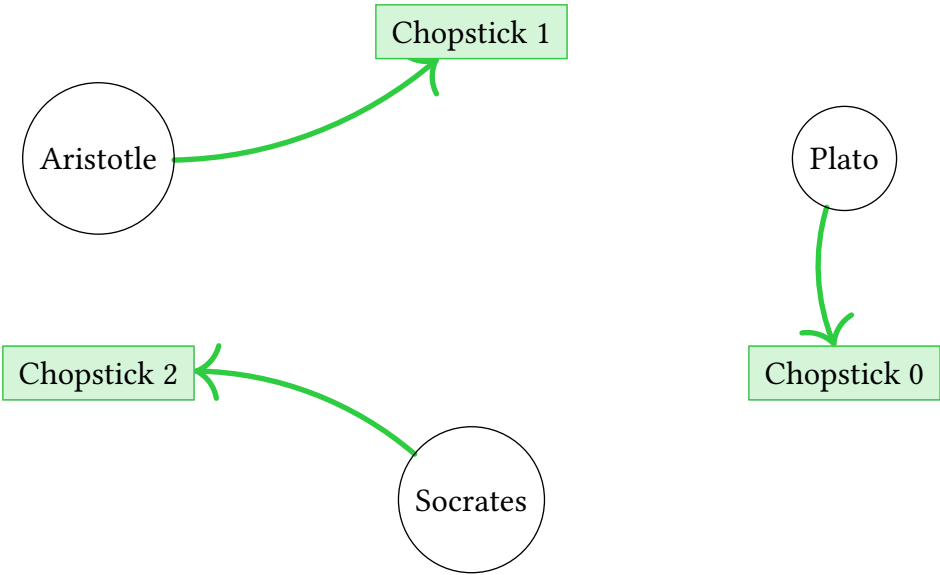


9.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

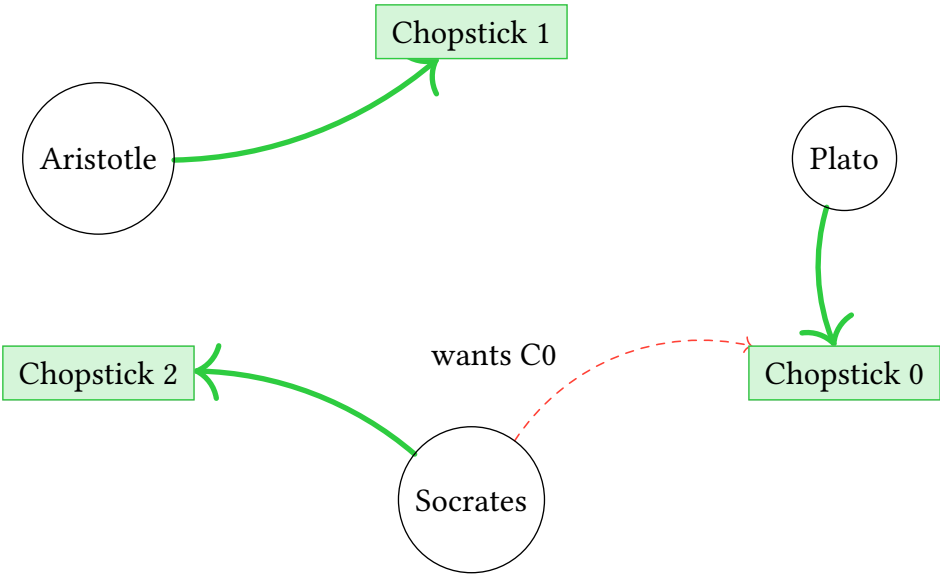


9.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

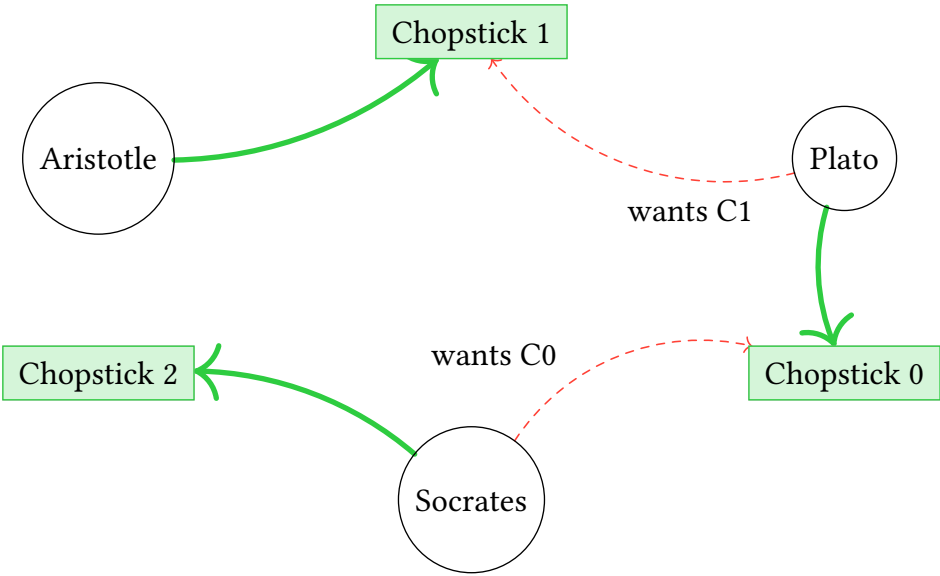


9.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

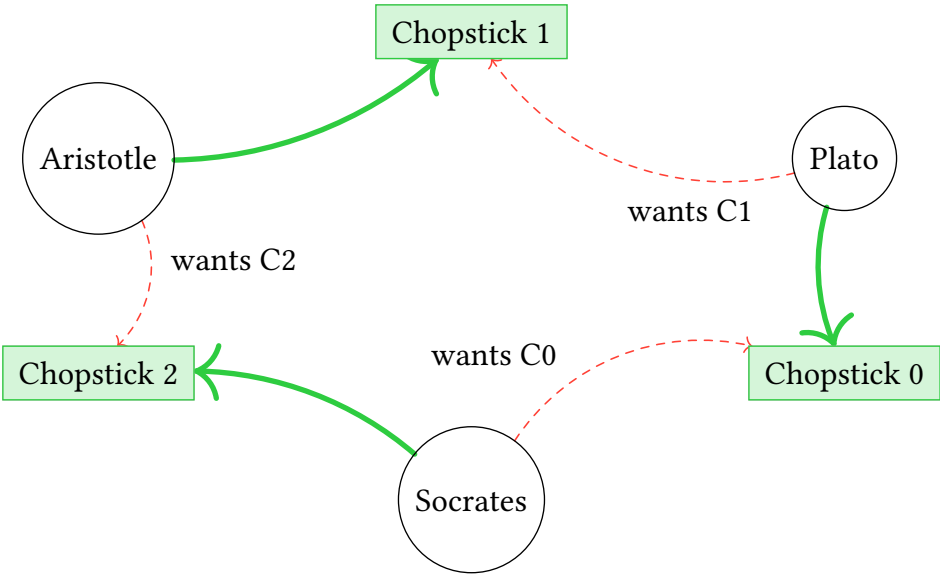


9.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

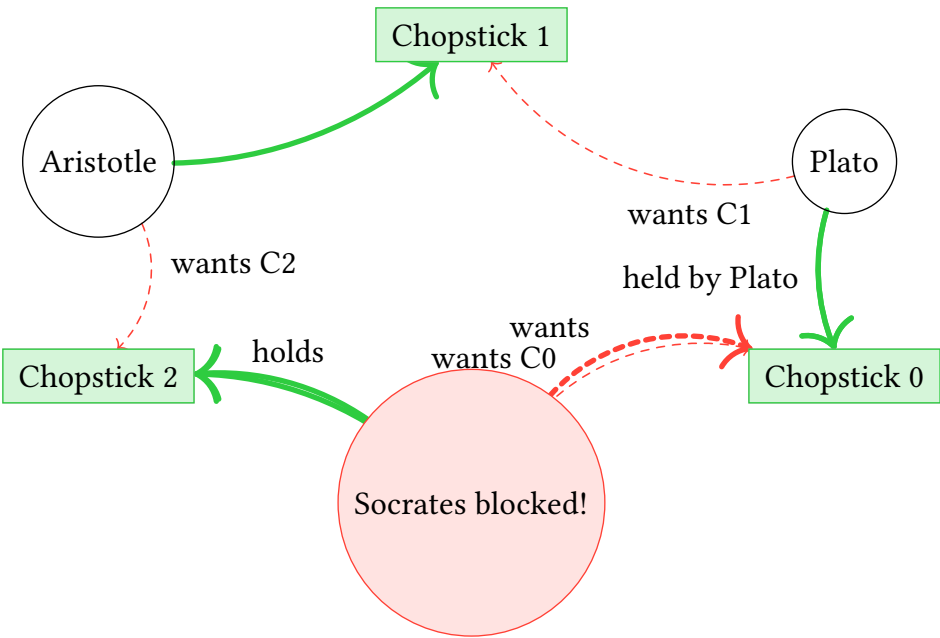


9.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

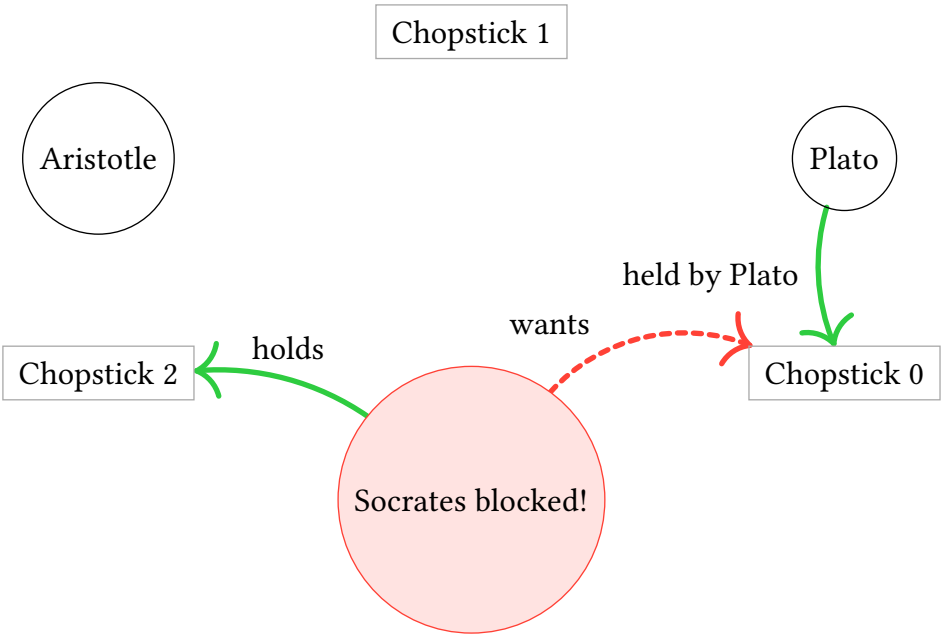
- Trying to grab
- Holding successfully
- - - Wants but blocked

9.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

- Trying to grab
- Holding successfully
- - - Wants but blocked

Socrates' perspective:

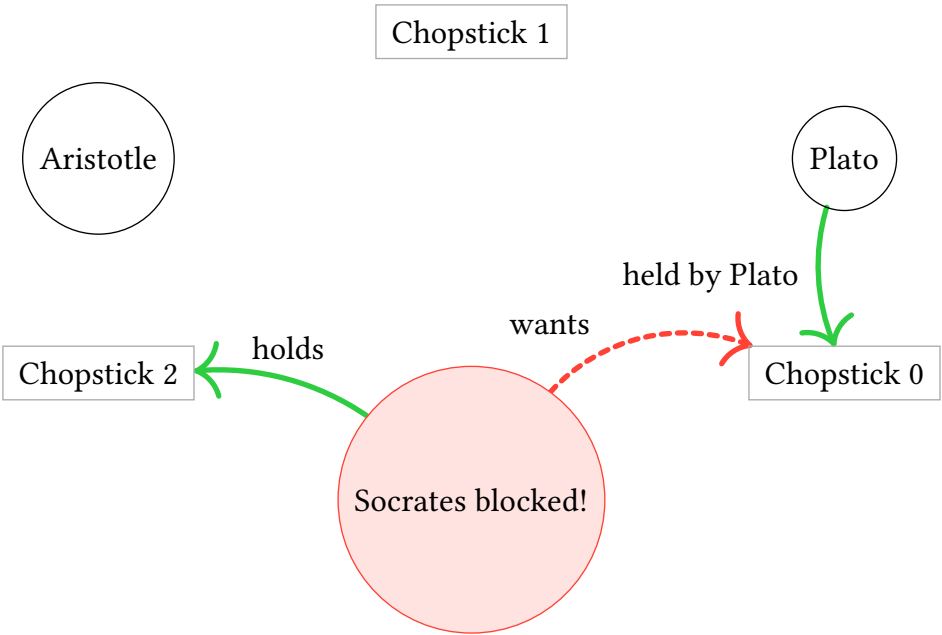
- Holds C2 (green)
- Wants C0 (red dashed)
- But C0 is held by Plato!

9.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

- Trying to grab
- Holding successfully
- - - Wants but blocked

Socrates' perspective:

- Holds C2 (green)
- Wants C0 (red dashed)
- But C0 is held by Plato!

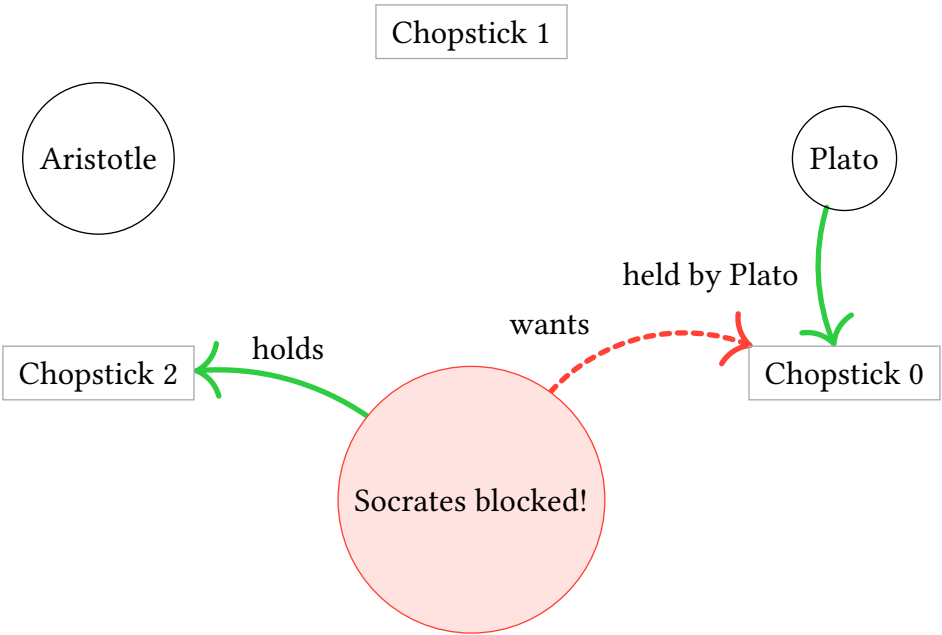
Same for Plato and Aristotle → circular wait → **Deadlock!**

9.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

- Trying to grab
- Holding successfully
- - - Wants but blocked

Socrates' perspective:

- Holds C2 (green)
- Wants C0 (red dashed)
- But C0 is held by Plato!

Same for Plato and Aristotle → circular wait → **Deadlock!**

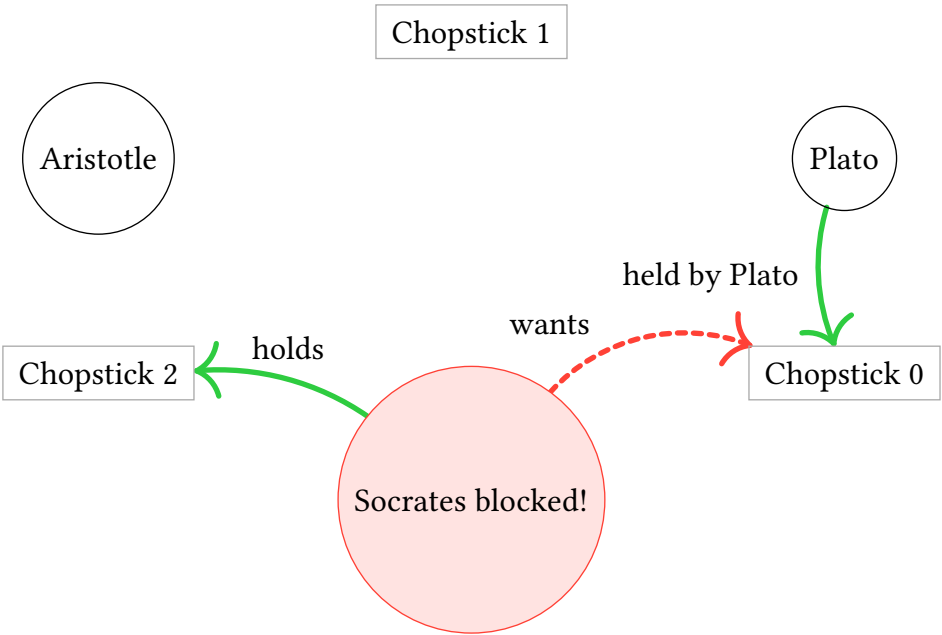
Which Coffman condition should we break?

9.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

- Trying to grab
- Holding successfully
- - - Wants but blocked

Socrates' perspective:

- Holds C2 (green)
- Wants C0 (red dashed)
- But C0 is held by Plato!

Same for Plato and Aristotle → circular wait → **Deadlock!**

Which Coffman condition should we break?

Circular wait: change lock order for one philosopher.

9.2. Assignment

Solve the “Dining Philosophers” problem with async tasks instead of threads.

- Revisit the standard thread-based version in `session6/examples/philosophers.rs`
- Complete exercise code in `session7/examples/philosophers.rs`.

Solutions are provided as `*-solution.rs` files.

9.3. Additional bonus

9. Exercise: async philosophers

Do we still need the asymmetry (the last philosopher switching hands)?

Do we still need the asymmetry (the last philosopher switching hands)?

Yes, because all philosophers are still working in parallel in separate tasks, so we may have a deadlock.

How to solve this problem with only a single async task? Do we still have deadlocks

Do we still need the asymmetry (the last philosopher switching hands)?

Yes, because all philosophers are still working in parallel in separate tasks, so we may have a deadlock.

How to solve this problem with only a single async task? Do we still have deadlocks

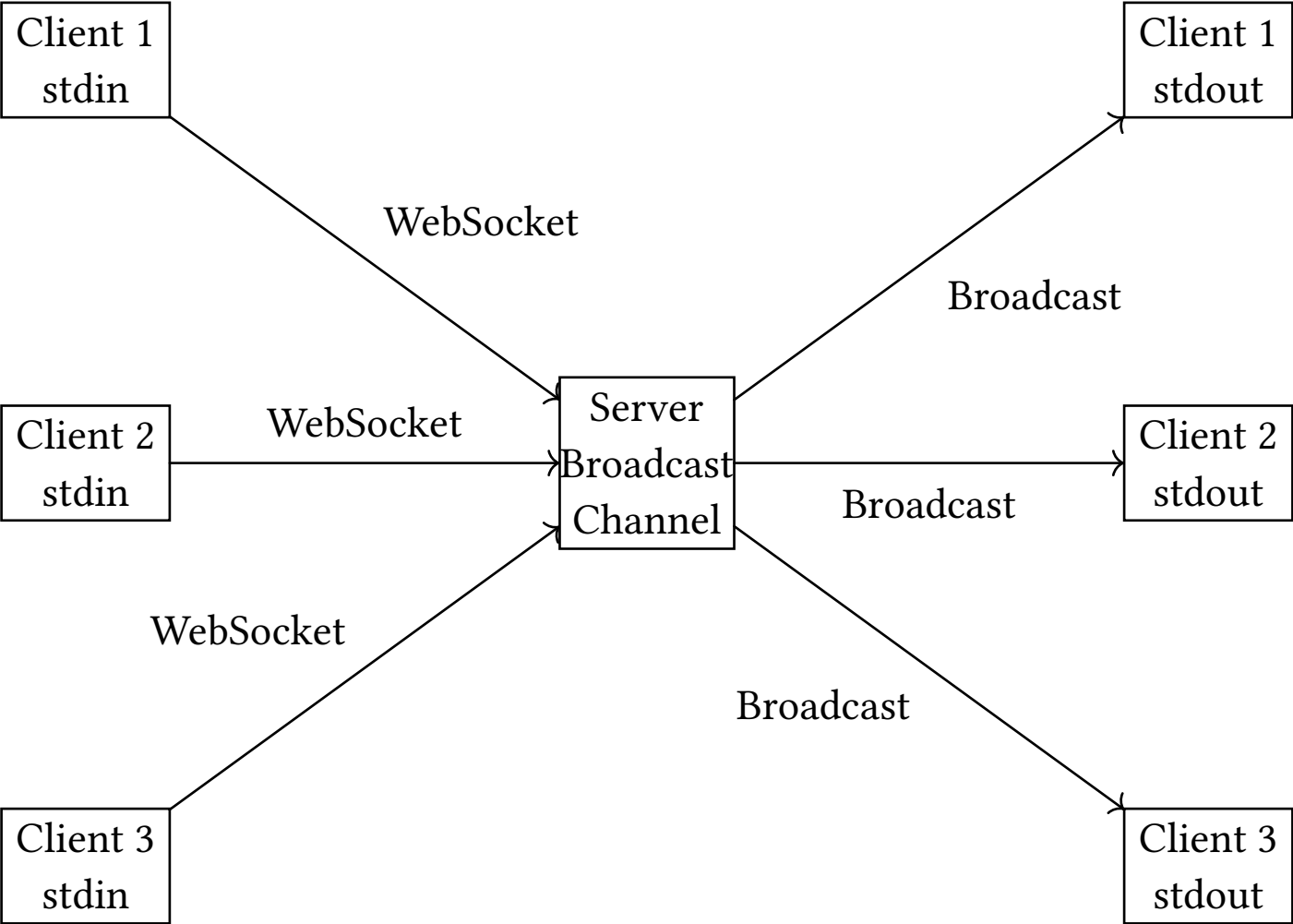
No, since philosophers access sequentially.

1.	Student presentations	1
2.	Asynchronous programming	4
3.	Asynchronous Rust	8
4.	Rust's async implementation	15
5.	Ecosystem	22
6.	Common async datatypes and functions	28
7.	Streams	38
8.	Pitfalls	44
9.	Exercise: async philosophers	49
10.	Exercise: broadcast chat application	53
10.1.	Overview	54
10.2.	APIs	56
10.3.	Binaries	57
10.4.	Tasks	58

10.1. Overview

In this exercise, we want to use our new knowledge to implement a broadcast chat application. We have a chat server that the clients connect to and publish their messages. The client reads user messages from the standard input, and sends them to the server. The chat server broadcasts each message that it receives to all the clients.

For this, we use a broadcast channel on the server, and `tokio_websockets` for the communication between the client and the server.



10.2. APIs

You are going to need the following functions from `tokio` and `tokio_websockets`. Spend a few minutes to familiarize yourself with the API.

From the `futures` (or `futures-util`) crate:

- `StreamExt::next()` implemented by `WebSocketStream`: for asynchronously reading messages from a Websocket Stream.
- `SinkExt::send()` implemented by `WebSocketStream`: for asynchronously sending messages on a Websocket Stream.

Specific to Tokio:

- `Lines::next_line()`: for asynchronously reading user messages from the standard input. https://docs.rs/tokio/latest/tokio/io/struct.Lines.html#method.next_line
- `Sender::subscribe()`: for subscribing to a broadcast channel. <https://docs.rs/tokio/latest/tokio/sync/broadcast/struct.Sender.html#method.subscribe>

10.3. Binaries

Normally in a Cargo project, you can have only one binary, and one `src/main.rs` file.

In this project, we need two binaries: one for the client, and one for the server.

You could potentially make them two separate Cargo projects, but we are going to put them in a single Cargo project with two binaries.

For this to work, the client and the server code are in `session7/src/bin`.

Run the server with:

```
1 cargo run --bin server
```

[\(playground link\)](#)

and the client with:

```
1 cargo run --bin client
```

[\(playground link\)](#)

10.4. Tasks

10.4.1. Server

Implement the `handle_connection` function in `src/bin/server.rs`.

Hint: Use `tokio::select!` for concurrently performing two tasks in a continuous loop:

- One task receives messages from the client and broadcasts them.
- The other sends messages received by the server to the client.

10.4.2. Client

Complete the `main` function in `src/bin/client.rs`.

Hint: As before, use `tokio::select!` in a continuous loop for concurrently performing two tasks:

1. reading user messages from standard input and sending them to the server, and
2. receiving messages from the server, and displaying them for the user.

Optional: Once you are done, change the code to broadcast messages to all clients, but the sender of the message.